C3103

Computer Networks



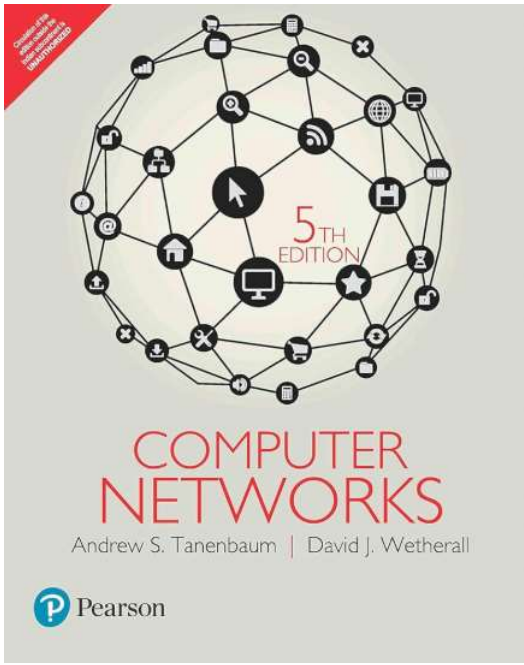
Dr. Pradeepthi K V

C R Rao Advanced Institute of Mathematics, Statistics and Computer Science

University of Hyderabad Campus, Gachibowli, Hyderabad.

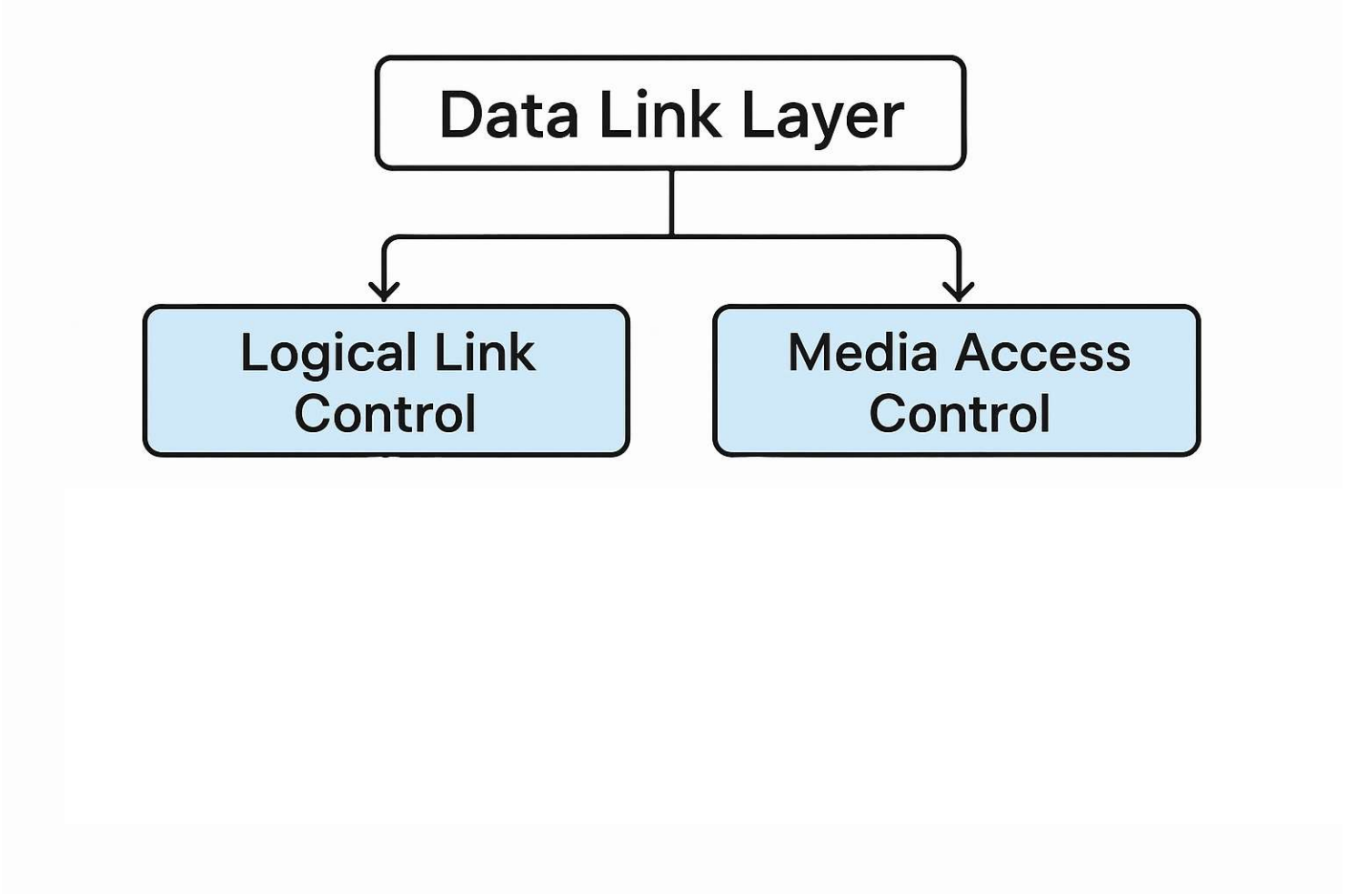
Unit-2

- Elementary data link protocols: Simplex protocol, A simplex stop and wait protocol for an error-free channel, A simplex stop and wait protocol for noisy channel.
- Sliding Window protocols: A one-bit sliding window protocol, A protocol using Go-Back-N, A protocol using Selective Repeat, Example data link protocols.
- Medium Access sublayer: The channel allocation problem, Multiple access protocols: ALOHA, Carrier sense multiple access protocols, collision free protocols. Wireless LANs, Data link layer switching.



- Reference:

Computer Networks, 5th Edition by Andrew Tanenbaum & David Wetherall, Copyright. Pearson Education 2011

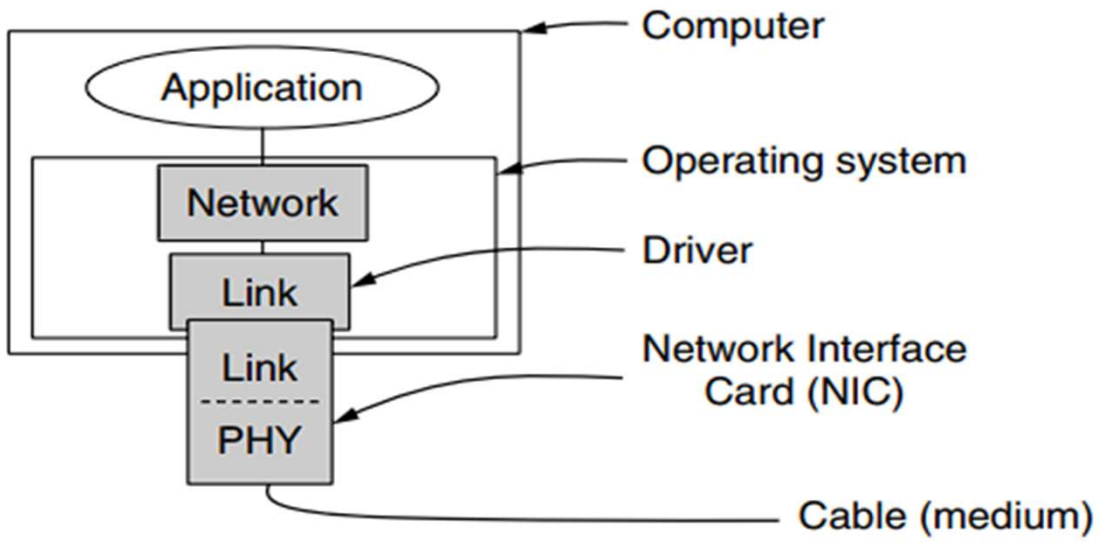

Data Link Layer
Logical Link Control
Media Access Control

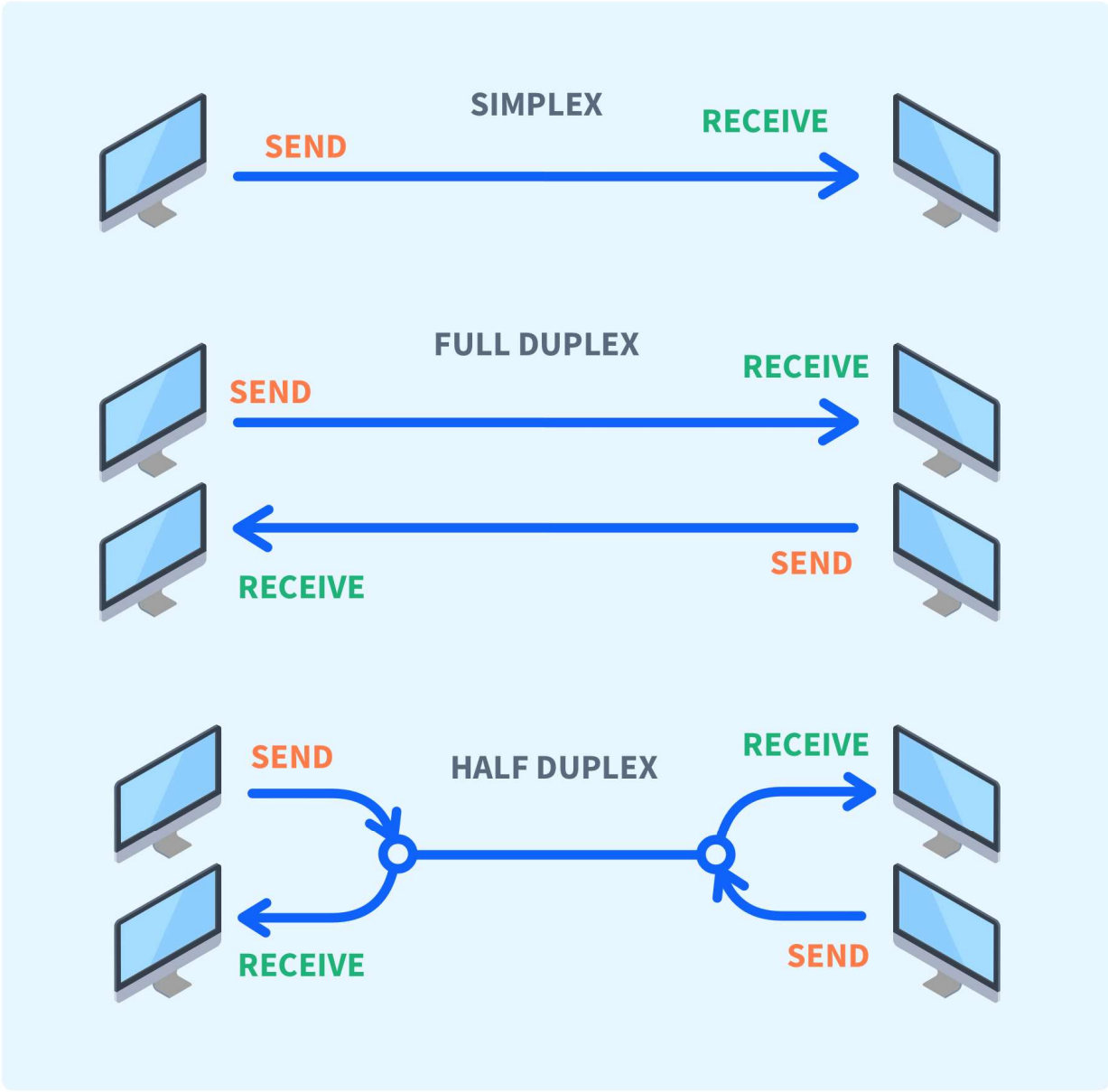
Elementary Data Link Protocols

- Utopian/Unrestricted Simplex Protocol
- Simplex Stop-and-Wait Protocol
  - Error-Free Channel
- Simplex Stop-and-Wait Protocol
  - Noisy Channel

Assumptions :

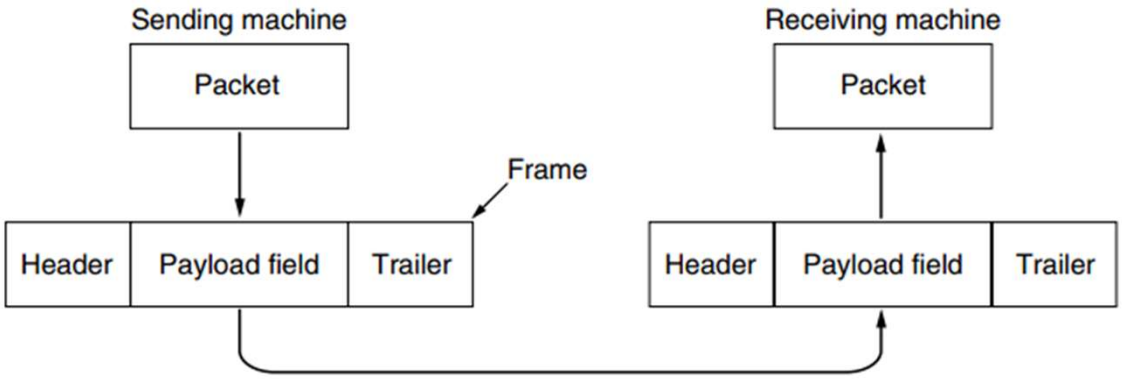
- The physical layer, data link layer, and network layer are independent processes that communicate by passing messages back and forth
- Machine **A** wants to send a long stream of data to machine **B,** using a reliable, connection oriented service.




SIMPLEX
SEND
RECEIVE
FULL DUPLEX
SEND
RECEIVE
RECEIVE
SEND
HALF DUPLEX
SEND
RECEIVE
RECEIVE
SEND

Data Link Layer:

- The packet passed across the interface to data link layer is pure data which is delivered to the destination's network layer.
- It encapsulates the packet in a frame by adding a data link header and trailer to it.



- We also assume that there exist suitable library procedures "to_physical_layer", "from_physical_layer" and "wait_for_event"

Some definitions needed in the protocols:

- Some common declarations that are common to many protocols are:

Five data structures

-> ***boolean:*** *Takes true or false*
-> ***seq_nr:*** *Small integer used to number the frames so that we can tell them apart. Sequence numbers run from 0 up to and including MAX SEQ, which is defined in each protocol needing it*
-> ***packet:*** *Unit of information exchanged between the network layer and the data link layer on the same machine, or between network layer peers.*
-> ***frame_kind:*** *Kind field tells whether there are any data in the frame, because some of the protocols distinguish frames containing only control information from those containing data as well.*

Frame :

Frame is composed of four fields :

i) kind
ii) Seq
iii) ack
iv) info

} **Frame Header**

➔ The seq and ack fields are used for sequence numbers and acknowledgements.

➔ The info field of a data frame contains a single packet; the info of a control frame is not used.

```
#define MAX_PKT 1024                                /* determines packet size in bytes */

typedef enum {false, true} boolean;                 /* boolean type */
typedef unsigned int seq_nr;                        /* sequence or ack numbers */
typedef struct {unsigned char data[MAX_PKT];} packet;/*    packet definition */
typedef enum {data, ack, nak} frame_kind;           /* frame_kind definition */

typedef struct {                                    /* frames are transported in this layer */
  frame_kind kind;                                  /* what kind of a frame is it? */
  seq_nr seq;                                       /* sequence number */
  seq_nr ack;                                       /* acknowledgement number */
  packet info;                                      /* the network layer packet */
} frame;
```

```
/* Wait for an event to happen; return its type in event. */
void wait_for_event(event_type *event);

/* Fetch a packet from the network layer for transmission on the channel. */
void from_network_layer(packet *p);

/* Deliver information from an inbound frame to the network layer. */
void to_network_layer(packet *p);

/* Go get an inbound frame from the physical layer and copy it to r. */
void from_physical_layer(frame *r);

/* Pass the frame to the physical layer for transmission. */
void to_physical_layer(frame *s);

/* Start the clock running and enable the timeout event. */
void start_timer(seq_nr k);

/* Stop the clock and disable the timeout event. */
void stop_timer(seq_nr k);

/* Start an auxiliary timer and enable the ack_timeout event. */
void start_ack_timer(void);

/* Stop the auxiliary timer and disable the ack_timeout event. */
void stop_ack_timer(void);

/* Allow the network layer to cause a network_layer_ready event. */
void enable_network_layer(void);

/* Forbid the network layer from causing a network_layer_ready event. */
void disable_network_layer(void);

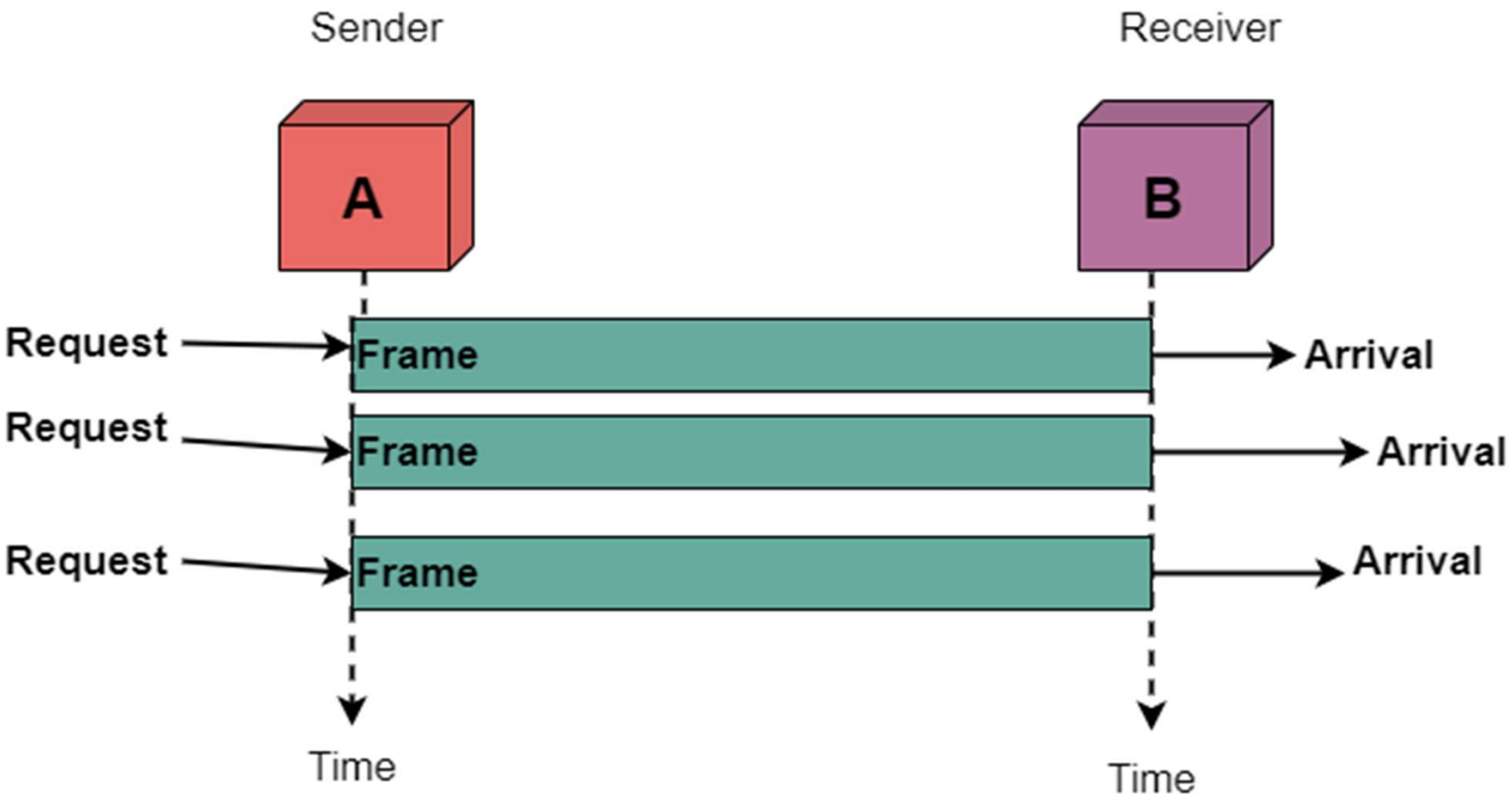
/* Macro inc is expanded in-line: Increment k circularly. */
#define inc(k) if (k < MAX_SEQ) k = k + 1; else k = 0
```

Elementary Data Link Protocols

- Unrestricted Simplex Protocol
- Simplex Stop-and-Wait Protocol
- Simplex Protocol for a Noisy Channel

Unrestricted Simplex Protocol :

- Data transmitting is carried out in one direction only.
- Both the transmitted and receiving network layers are always ready.
- Processing time can be ignored, infinite buffer space is available and best of all, the communication channel between the data link layers never damages or loses of frames.
- This thoroughly unrealistic protocol is nicknamed “Utopia”.


Sender
Receiver
A
B
Request
Frame
Arrival
Request
Frame
Arrival
Request
Frame
Arrival
Time
Time





```
/* Protocol 1 (utopia) provides for data transmission in one direction only, from
   sender to receiver.  The communication channel is assumed to be error free,
   and the receiver is assumed to be able to process all the input infinitely quickly.
   Consequently, the sender just sits in a loop pumping data out onto the line as
   fast as it can. */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender1(void)
{
  frame s;                              /* buffer for an outbound frame */
  packet buffer;                        /* buffer for an outbound packet */

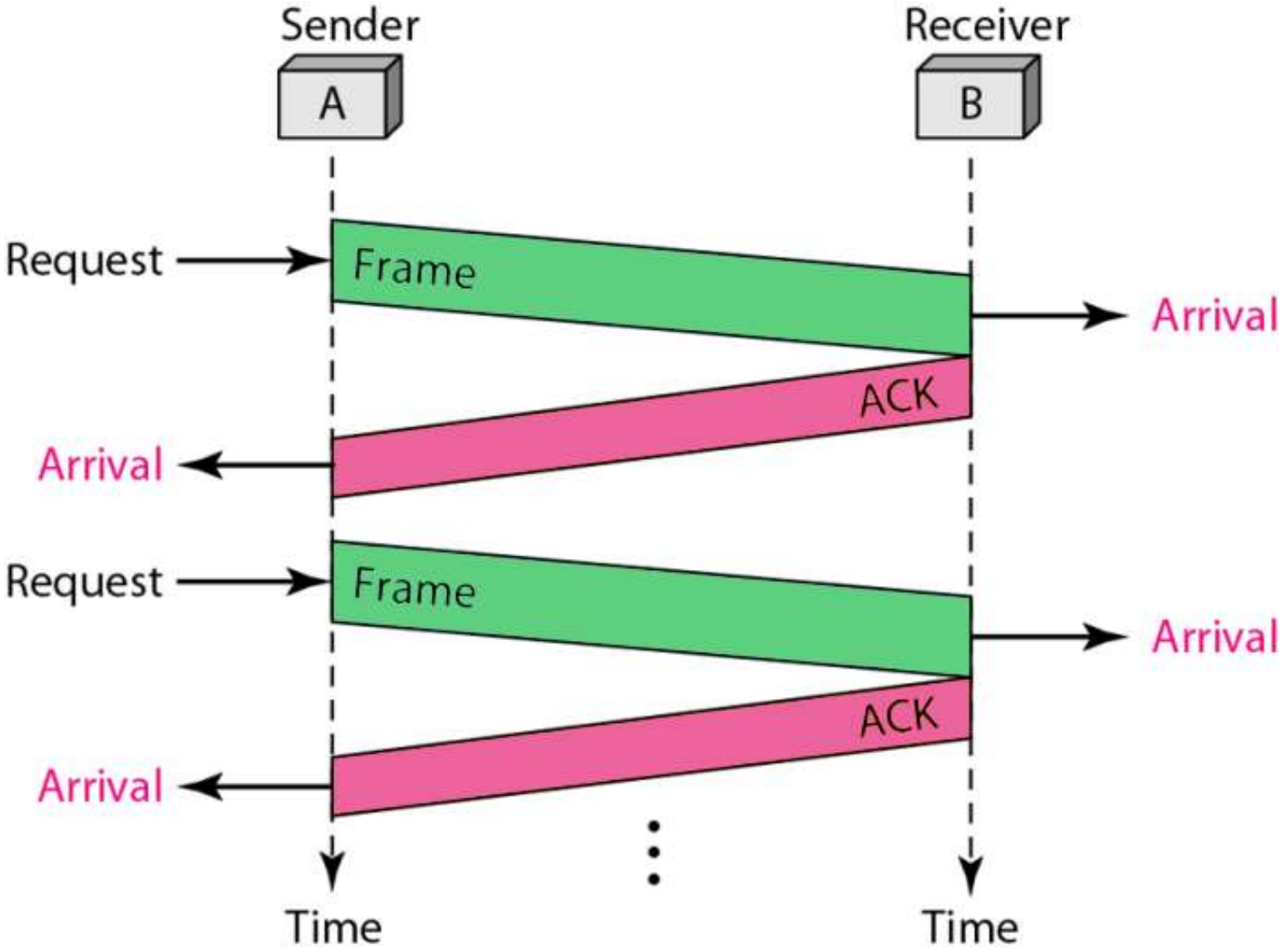
  while (true) {
      from_network_layer(&buffer);      /* go get something to send */
      s.info = buffer;                  /* copy it into s for transmission */
      to_physical_layer(&s);            /* send it on its way */
  }                                 /     * Tomorrow, and tomorrow, and tomorrow,
                                          Creeps in this petty pace from day to day
                                          To the last syllable of recorded time
                                              - Macbeth, V, v */
}

void receiver1(void)
{
  frame r;
  event_type event;                     /* filled in by wait, but not used here */

  while (true) {
      wait_for_event(&event);           /* only possibility is frame_arrival */
      from_physical_layer(&r);          /* go get the inbound frame */
      to_network_layer(&r.info);        /* pass the data to the network layer */
  }
}
```

Simplex Stop and Wait protocol (Error-Free Channel)

- This protocol assumes same conditions as in Simplex protocol but the receiver can only process the received information at finite rate.
- One solution is to build the receiver to be powerful enough to process a continuous stream of back-to-back frames.
- More general solution for this problem would be to have an acknowledgement sent by receiver after a frame arrives.
- This protocol is called Simplex Stop and wait protocol, the sender sends one frame and waits for feedback from the receiver. When the ACK arrives, the sender sends the next frame.


Sender
Receiver
A
B
Request
Frame
Arrival
ACK
Arrival
Request
Frame
Arrival
ACK
Arrival
Time
Time

```
/* Protocol 2 (stop-and-wait) also provides for a one-directional flow of data from
   sender to receiver. The communication channel is once again assumed to be error
   free, as in protocol 1. However, this time, the receiver has only a finite buffer
   capacity and a finite processing speed, so the protocol must explicitly prevent
   the sender from flooding the receiver with data faster than it can be handled. */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender2(void)
{
  frame s;                              /* buffer for an outbound frame */
  packet buffer;                        /* buffer for an outbound packet */
  event_type event;                     /* frame_arrival is the only possibility */

  while (true) {
      from_network_layer(&buffer);      /* go get something to send */
      s.info = buffer;                  /* copy it into s for transmission */
      to_physical_layer(&s);            /* bye bye little frame */
      wait_for_event(&event);           /* do not proceed until given the go ahead */
  }
}

void receiver2(void)
{
  frame r, s;                           /* buffers for frames */
  event_type event;                     /* frame_arrival is the only possibility */
  while (true) {
      wait_for_event(&event);           /* only possibility is frame_arrival */
      from_physical_layer(&r);          /* go get the inbound frame */
      to_network_layer(&r.info);        /* pass the data to the network layer */
      to_physical_layer(&s);            /* send a dummy frame to awaken sender */
  }
}
```

A Simplex Stop and Wait Protocol for a Noisy Channel:

- This protocol considers the normal situation of a communication channel that makes errors where Frames may be either damaged or lost completely.

- A variation of protocol 2: adding a timer will not work as it has a fatal flaw in it.

- Sender puts a sequence number in the header of each frame it sends.

- Size of the sequence number can be anything ranging from a bit to multiple bytes, it must satisfy these conditions
    i) Large enough for the protocol to work correctly.
    ii) It shouldn't be too much for the protocol.

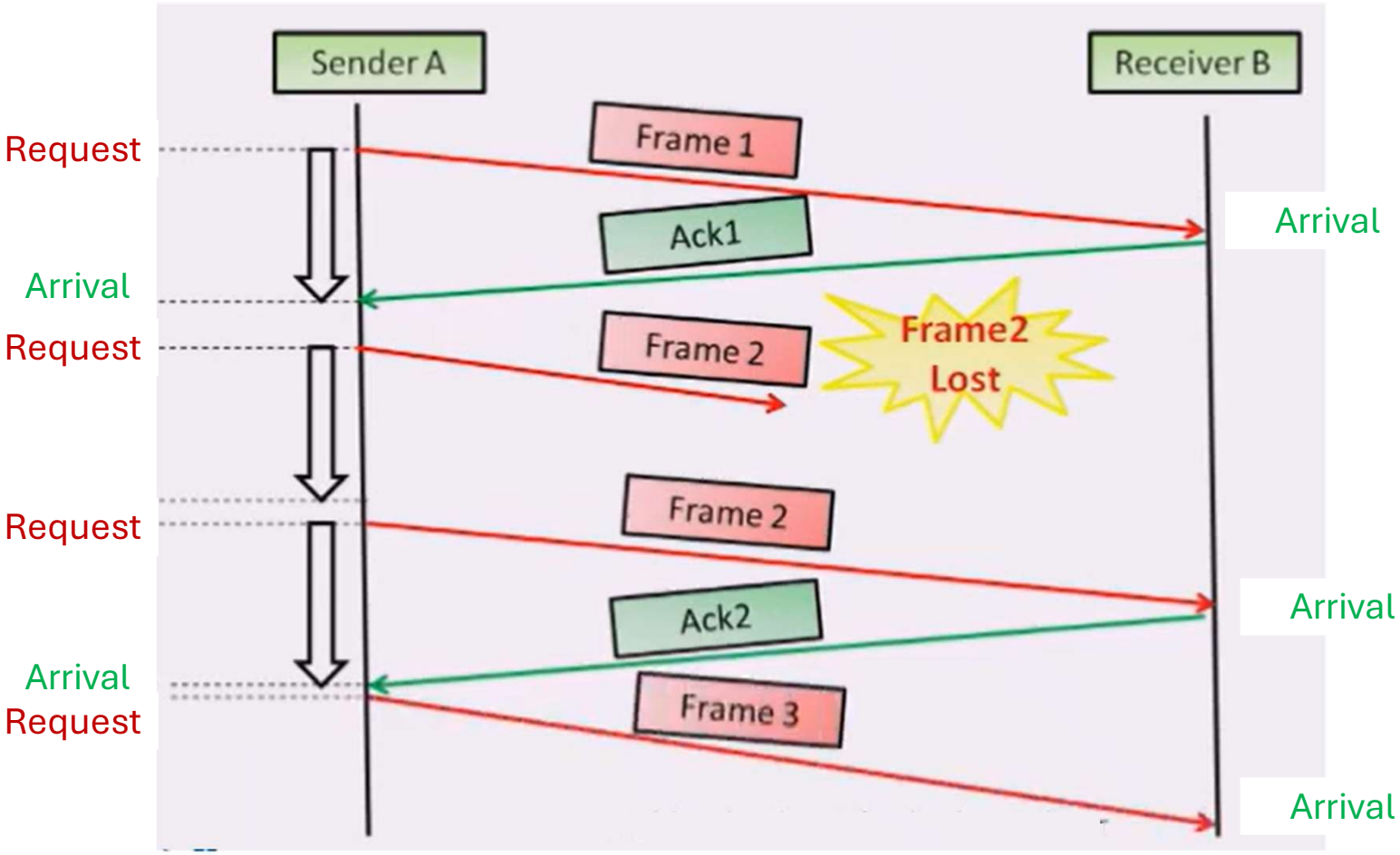
- The only ambiguity is between a frame and its immediate predecessor or successor

A Simplex Stop and Wait Protocol for a Noisy Channel Continued:

- A 1-bit sequence number(0 or 1) is sufficient.

- When a frame containing the correct sequence number arrives, it is accepted and passed to the network layer, then acknowledged. Then the expected sequence number is incremented modulo 2 (i.e., 0 becomes 1 and 1 becomes 0).

- Protocols in which the sender waits for a positive acknowledgement before advancing to the next data item are often called **ARQ (Automatic Repeat reQuest)** or **PAR (Positive Acknowledgement with Retransmission)**


Sender A
Receiver B
Request
Frame 1
Arrival
Ack1
Arrival
Request
Frame 2
Frame2
Lost
Request
Frame 2
Arrival
Ack2
Arrival
Request
Frame 3
Arrival

```
/* Protocol 3 (par) allows unidirectional data flow over an unreliable channel. */

#define MAX_SEQ 1                                   /* must be 1 for protocol 3 */
typedef enum  {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
  seq_nr next_frame_to_send;                        /* seq number of next outgoing frame */
  frame s;                                          /* scratch variable */
  packet buffer;                                    /* buffer for an outbound packet */
  event_type event;

  next_frame_to_send = 0;                           /* initialize outbound sequence numbers */
  from_network_layer(&buffer);                      /* fetch first packet */
  while (true) {
       s.info = buffer;                             /* construct a frame for transmission */
       s.seq = next_frame_to_send;                  /* insert sequence number in frame */
       to_physical_layer(&s);                       /* send it on its way */
       start_timer(s.seq);                          /* if answer takes too long, time out */
       wait_for_event(&event);                      /* frame_arrival, cksum_err, timeout */
       if (event == frame_arrival) {
            from_physical_layer(&s);                /* get the acknowledgement */
            if (s.ack == next_frame_to_send) {
                 stop_timer(s.ack);                 /* turn the timer off */
                 from_network_layer(&buffer);       /* get the next one to send */
                 inc(next_frame_to_send);           /* invert next_frame_to_send */
            }
       }
  }
}
```

```
void receiver3(void)
{
  seq_nr frame_expected;
  frame r, s;
  event_type event;

  frame_expected = 0;
  while (true) {
       wait_for_event(&event);                  /* possibilities: frame_arrival, cksum_err */
       if (event == frame_arrival) {            /* a valid frame has arrived. */
            from_physical_layer(&r);            /* go get the newly arrived frame */
            if (r.seq == frame  expected) {     /* this is what we have been waiting for. */
                 to_network_layer(&r.info);     /* pass the data to the network layer */
                 inc(frame_expected);           /* next time expect the other sequence nr */
            }
            s.ack = 1 – frame_expected;         /* tell which frame is being acked */
            to_physical_layer(&s);              /* send acknowledgement */
       }
  }
}
```

Advantages of Simplex Stop-and-wait- Protocol

- Simple reliable transmission

Disadvantages of Simplex Stop-and-wait- Protocol

- Sender always waits for acknowledgement frame to send the next frame.
- If the acknowledgement frame gets corrupted, the sender waits indefinity.

Advantages of Simplex Stop-and-wait- Protocol for noisy channel

- Handles lost frames using timer

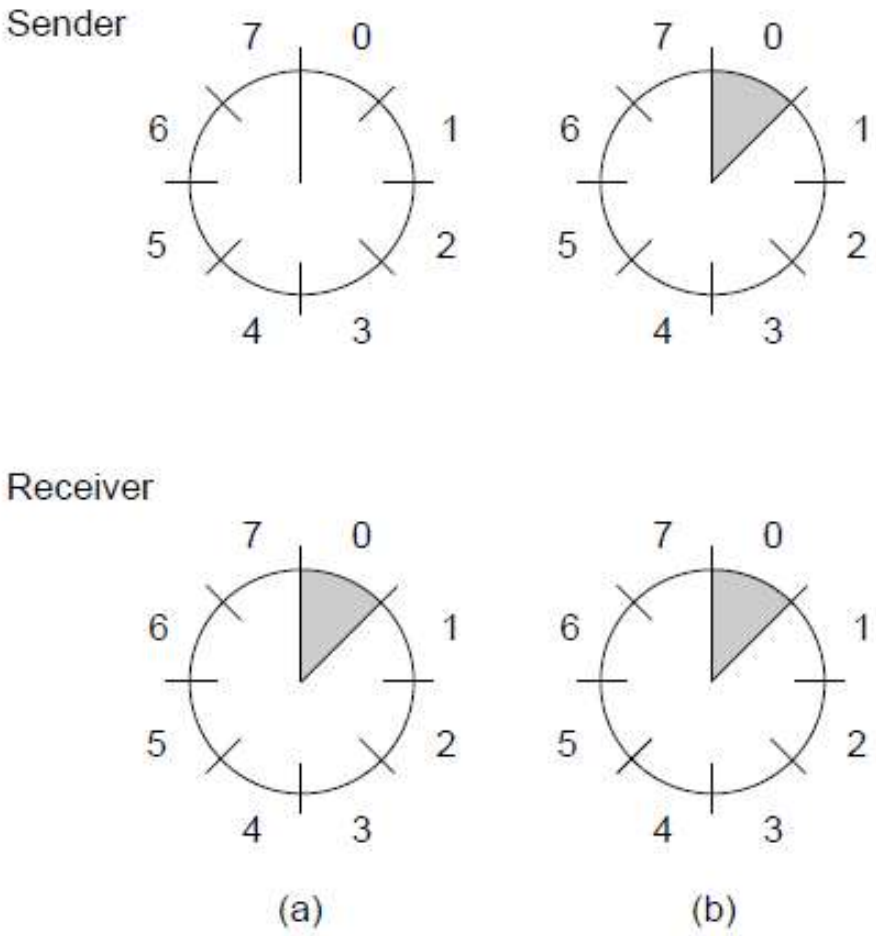
Disadvantages of Simplex Stop-and-wait- Protocol for nosy channel

- If the interval is too short, retransmission occurs(unwanted)
- If the interval is too long, bandwidth is wasted as sender waits too ling before doing a re transmission

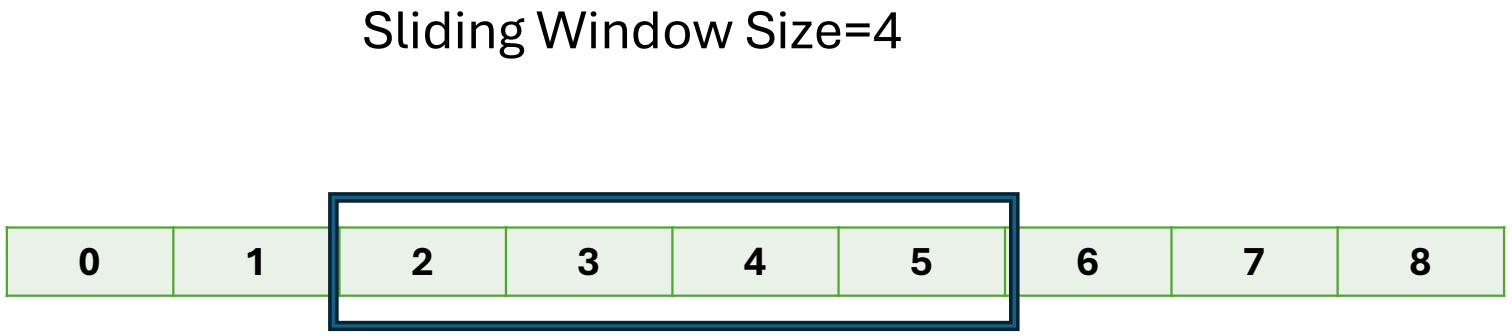
Sliding Window Protocols

A sliding window of size 1,

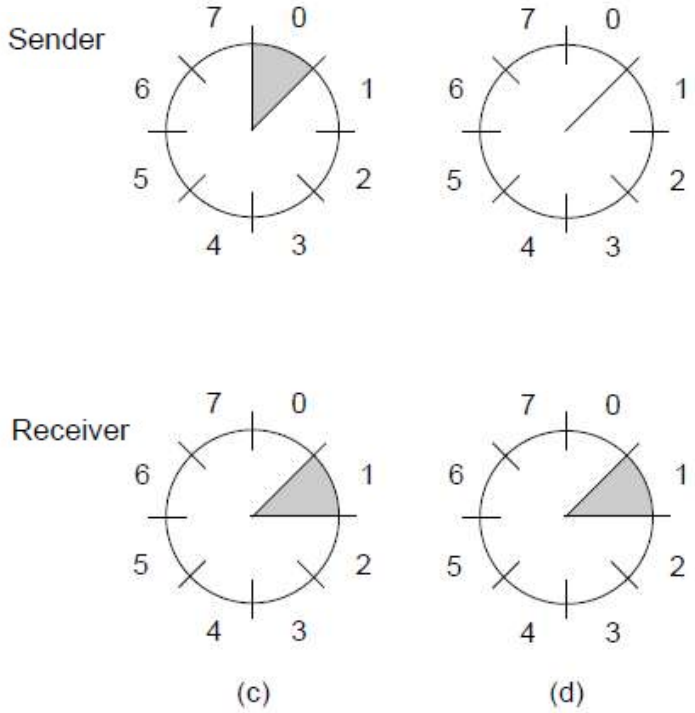
with a 3-bit sequence number.
(a) Initially. (b) After the first frame has been sent.



Sliding Window Protocols



Sliding Window Protocols



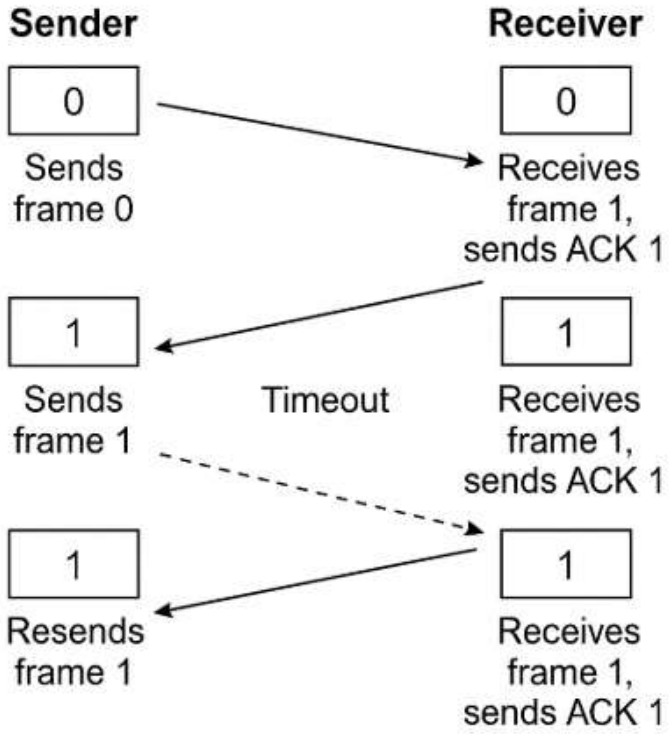
A sliding window of size 1, with a 3-bit sequence number

(c) After the first frame has been received. (d) After the first acknowledgement has been received.

Sliding Window Protocols

1. A one-bit sliding window Protocol
2. Go-Back N sliding window Protocol
3. Selective Repeat sliding window Protocol

A one-bit sliding window Protocol



One-Bit Sliding Window Protocol

A one-bit sliding window Protocol

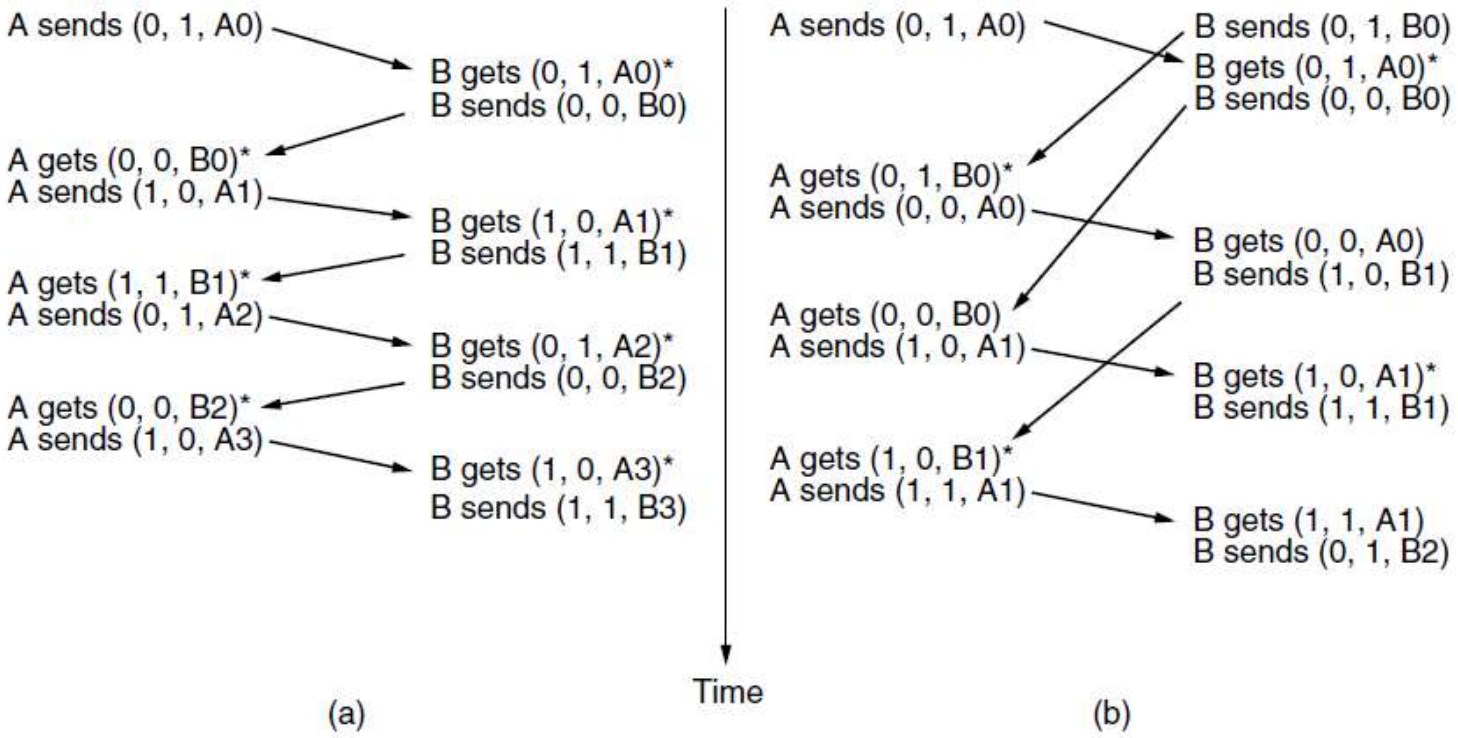
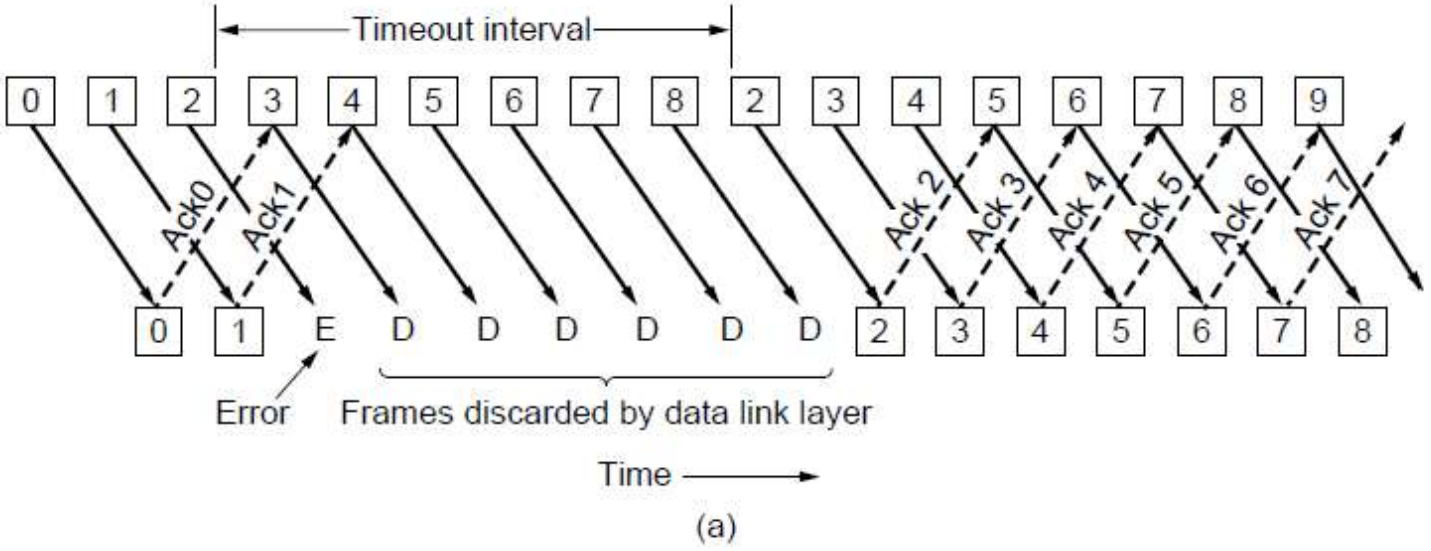


Figure 3-17. Two scenarios for protocol 4. (a) Normal case. (b) Abnormal case. The notation is (seq, ack, packet number). An asterisk indicates where a network layer accepts a packet.

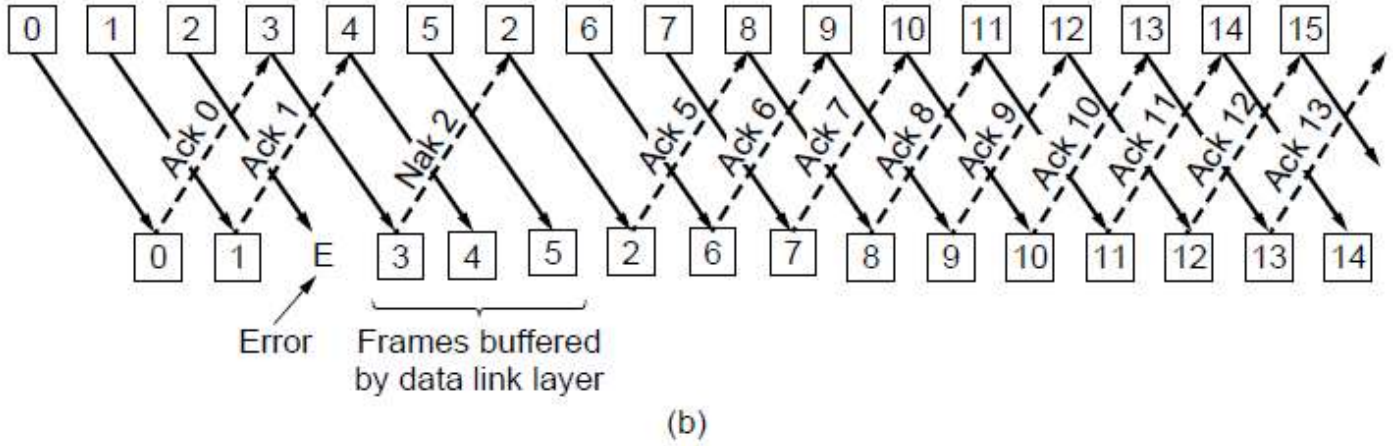
Protocol Using Go-Back-N

Pipelining and error recovery. Effect of an error when (a) receiver's window size is 1

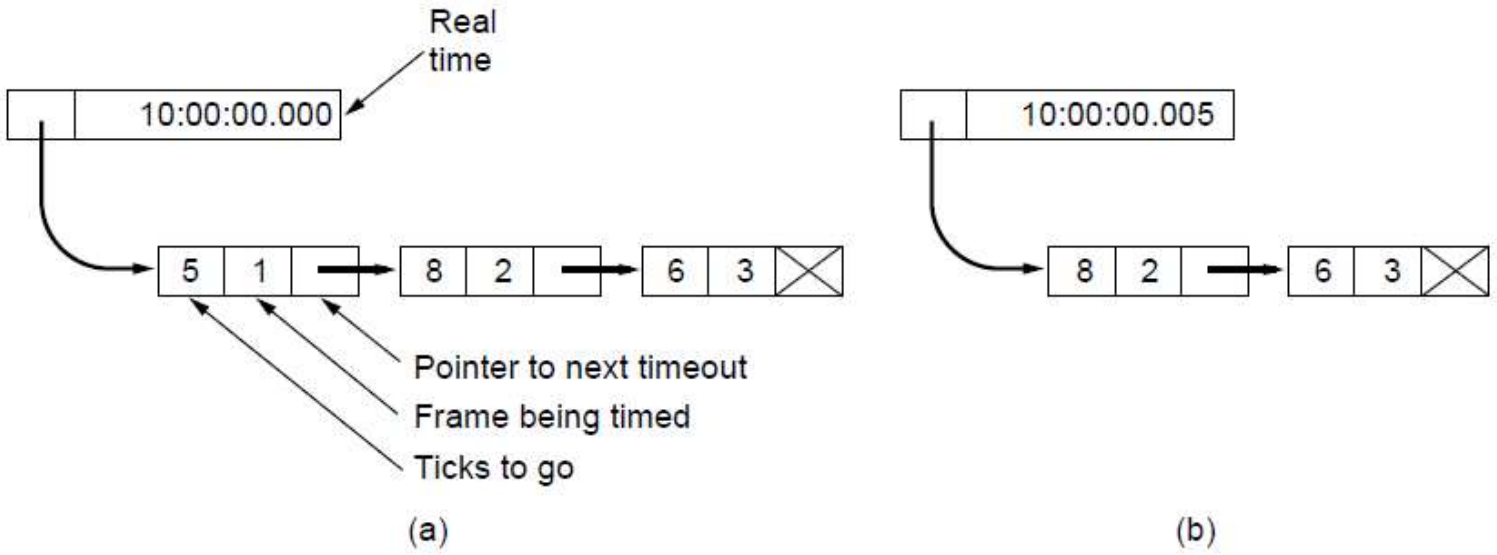


Protocol Using Go-Back-N

Pipelining and error recovery. Effect of an error when (b) receiver's window size is large.

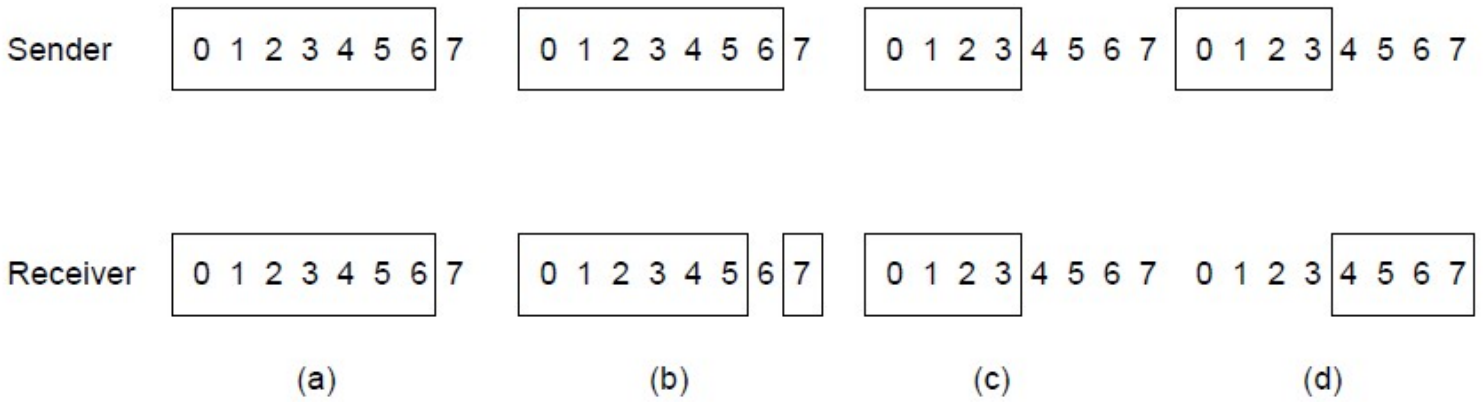


Protocol Using Go-Back-N



Simulation of multiple timers in software. (a) The queued timeouts (b) The situation after the first timeout has expired.

Protocol Using Selective Repeat



a)Initial situation with a window of size7

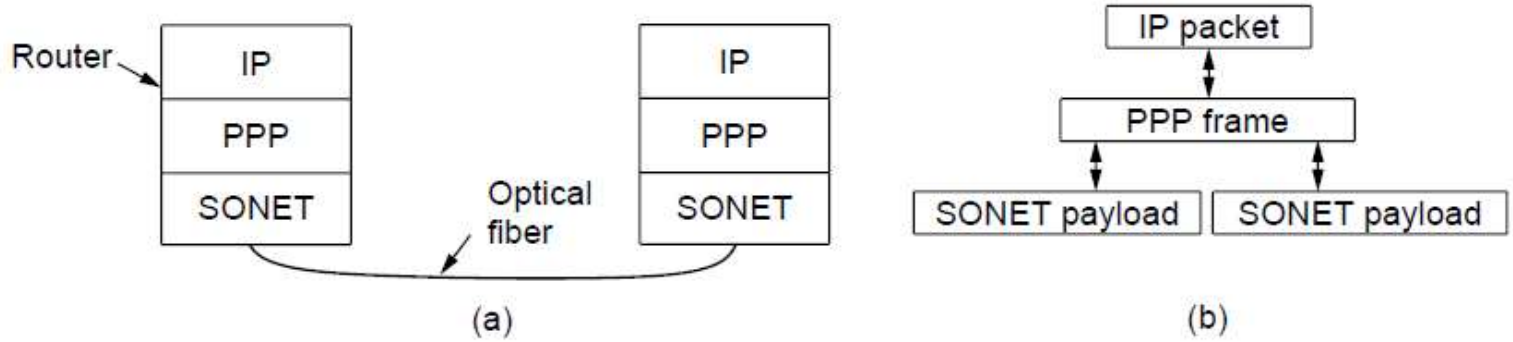
- After 7 frames sent and received but not acknowledged.
- Initial situation with a window size of 4.
- After 4 frames sent and received but not acknowledged.

Example Data Link Protocols

1. Packet over SONET
2. ADSL (Asymmetric Digital Subscriber Loop)

Packet over SONET (1)

Packet over SONET. (a) A protocol stack. (b) Frame relationships



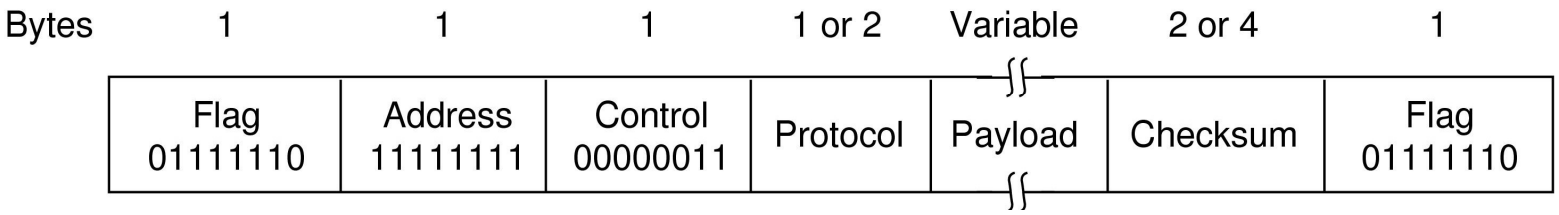
Packet over SONET (2)

PPP Features

1. Separate packets, error detection
2. Link Control Protocol
3. Network Control Protocol

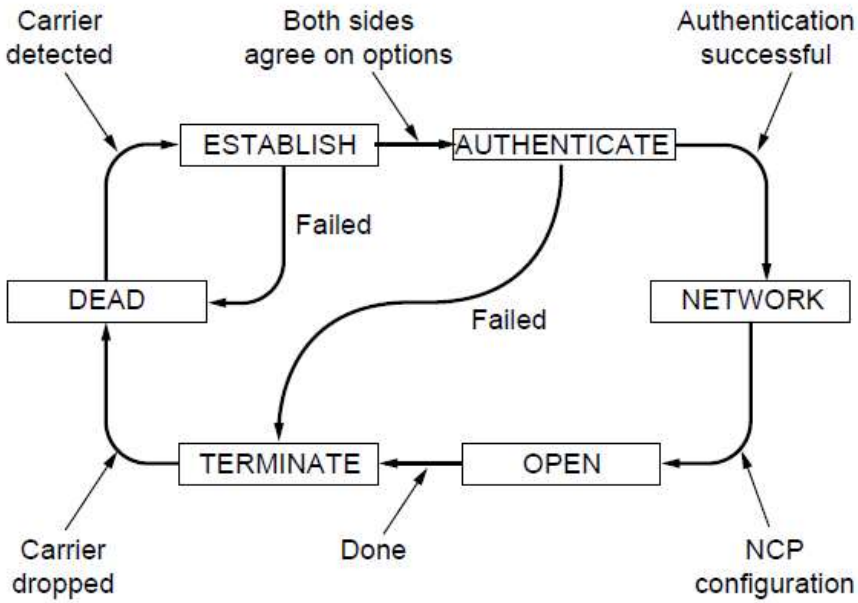
Packet over SONET (3)

The PPP full frame format for unnumbered mode operation

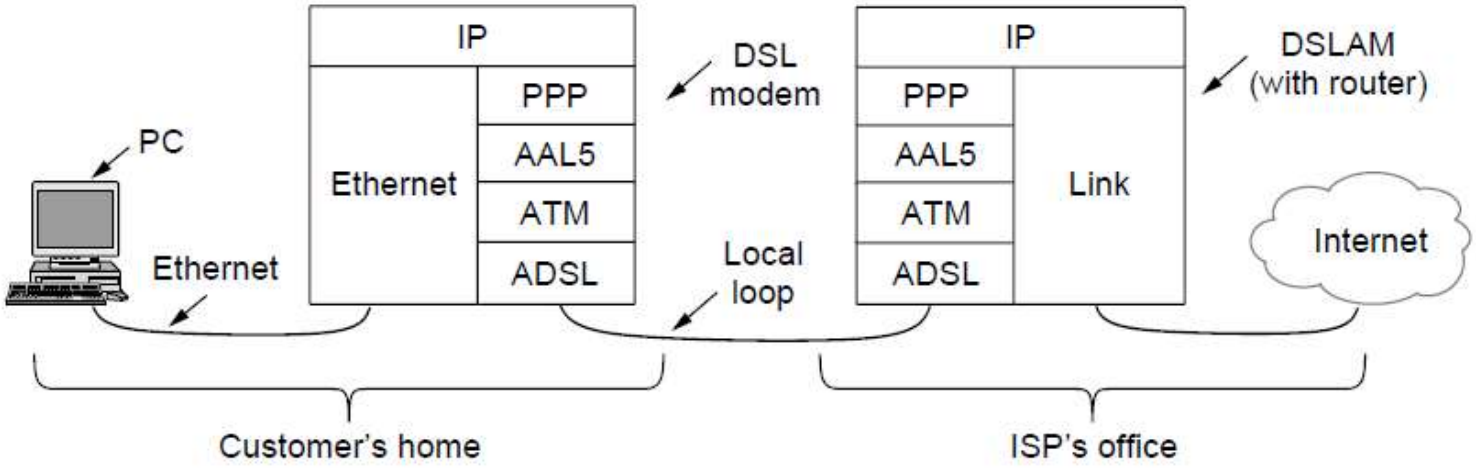


Packet over SONET (4)

State diagram for bringing a PPP link up and down

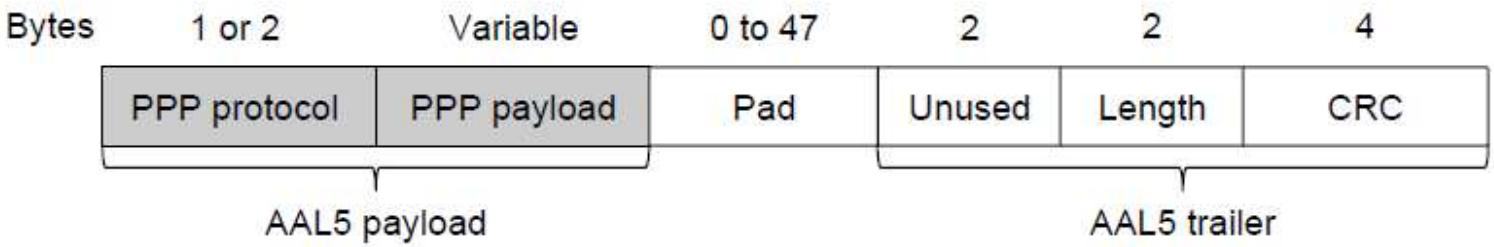


ADSL (Asymmetric Digital Subscriber Loop) (1)



ADSL (Asymmetric Digital Subscriber Loop) (1)

AAL5 frame carrying PPP data



Part 3: The Medium Access Control Sublayer

Outline

Part A Functions

1) The problem
2) Where are we now?
3) The Channel Allocation Problem

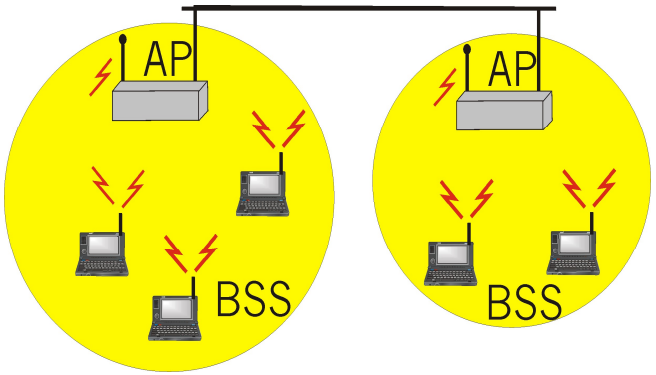
Part B Dynamic Channel Allocation Technologies

1) ALOHA Protocols
2) CSMA
3) CSMA/CD (old ETHERNET)
4) Switching (Fast ETHERNET)
5) Wireless LANS

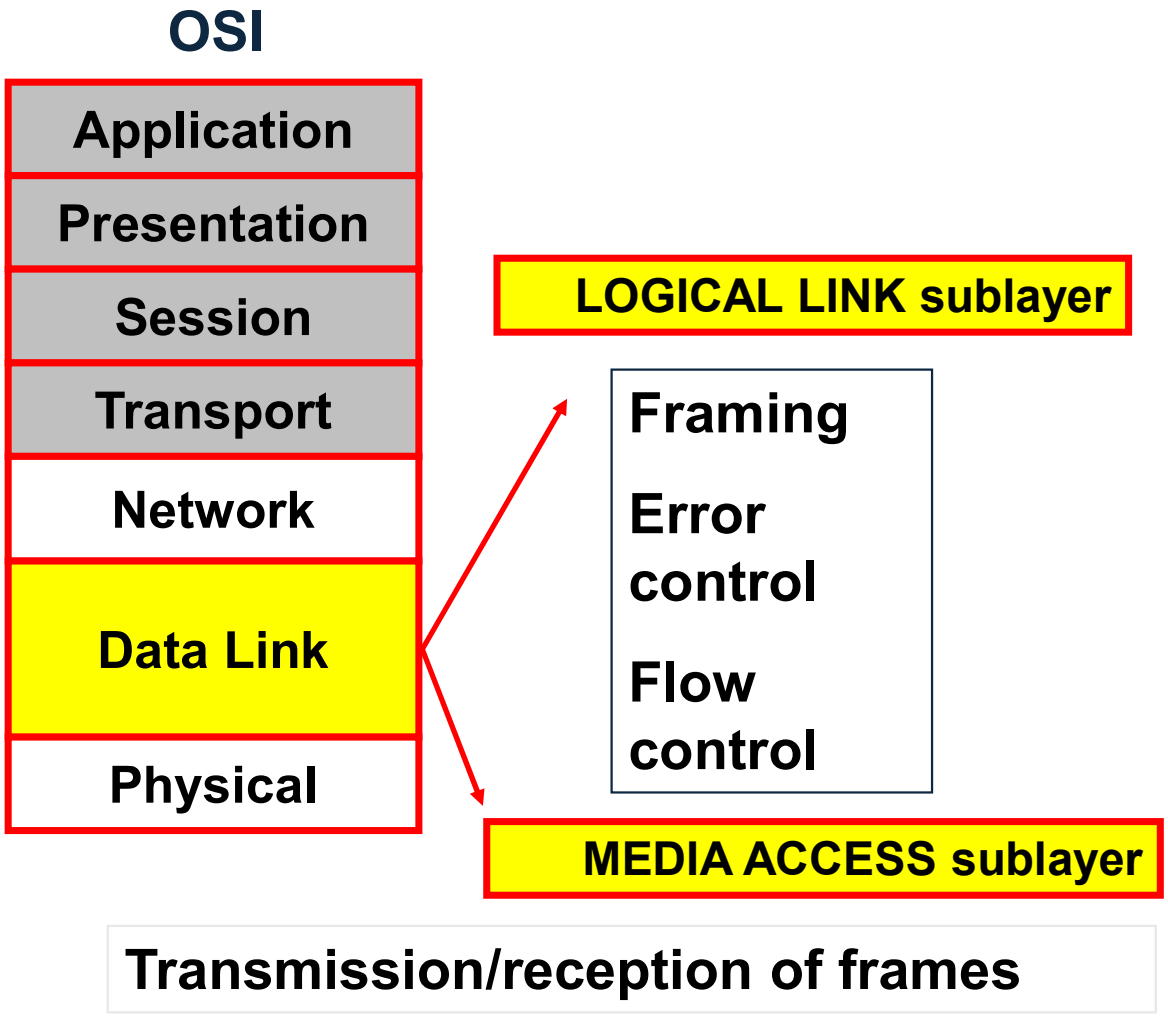
(1)The Problem

The problem: In a single channel broadcast network, when multiple stations try to send messages simultaneously, who has the right to use the channel?

A common sense:
When we take about MAC, we are faced with a broadcast network.



(2) Where are we now?



(3) The Channel Allocation Problem

- **Static**
  - FDM /TDM (Frequency/Time Division Multiplexing)
    - FDM : Radio/TV broadcasts
    - TDM : POTS (Plain Old Telephone System)
    - GSM uses both (Global System for Mobile Communications)
    - Wasteful of bandwidth and not work well for bursty traffic
- **Dynamic**
  - Pure/ Slotted ALOHA
  - Carrier Sense Multiple Access (CSMA) Protocols
  - Collision free protocols (optional contents)

Static Channel Allocation

- Static channel allocation refers to a **fixed method of dividing the communication channel** among users or devices **before any communication actually happens**. Each user gets a **dedicated portion** of the channel, regardless of whether or not they are actively using it.

Static Channel Allocation

Main Types:

- **Frequency Division Multiplexing (FDM):**
  - The channel is split into **multiple frequency bands**.
  - Each user is assigned a unique frequency.
  - Common in analog systems like FM radio.
- **Time Division Multiplexing (TDM):**
  - The channel is divided into **time slots**.
  - Each user gets access in a **round-robin** manner during their time slot.
  - Used in digital systems (like early telephone networks).

Static Channel Allocation

Assumptions:

- Static allocation is based on the idea that:
- The number of users is **fixed**.
- All users **generate traffic at the same average rate**.
- Channel capacity is **shared equally**, and traffic is **predictable**.

Static Channel Allocation

Problems with Static Allocation:

- **Inefficient use of bandwidth:**
  - If a user doesn’t have data to send, their allocated frequency or time slot goes unused.
- **Poor scalability:**
  - Adding new users is difficult without reshuffling the entire allocation.
- **Not suitable for bursty traffic:**
  - Real-world traffic is random and bursty, making fixed allocation wasteful.

Static Channel Allocation

Why Dynamic Allocation is Better:

- To overcome the limitations of static allocation, **dynamic channel allocation** schemes (like ALOHA, CSMA) allow stations to access the channel **on demand**, improving efficiency, especially when traffic is unpredictable.

Dynamic Channel Allocation Technologies

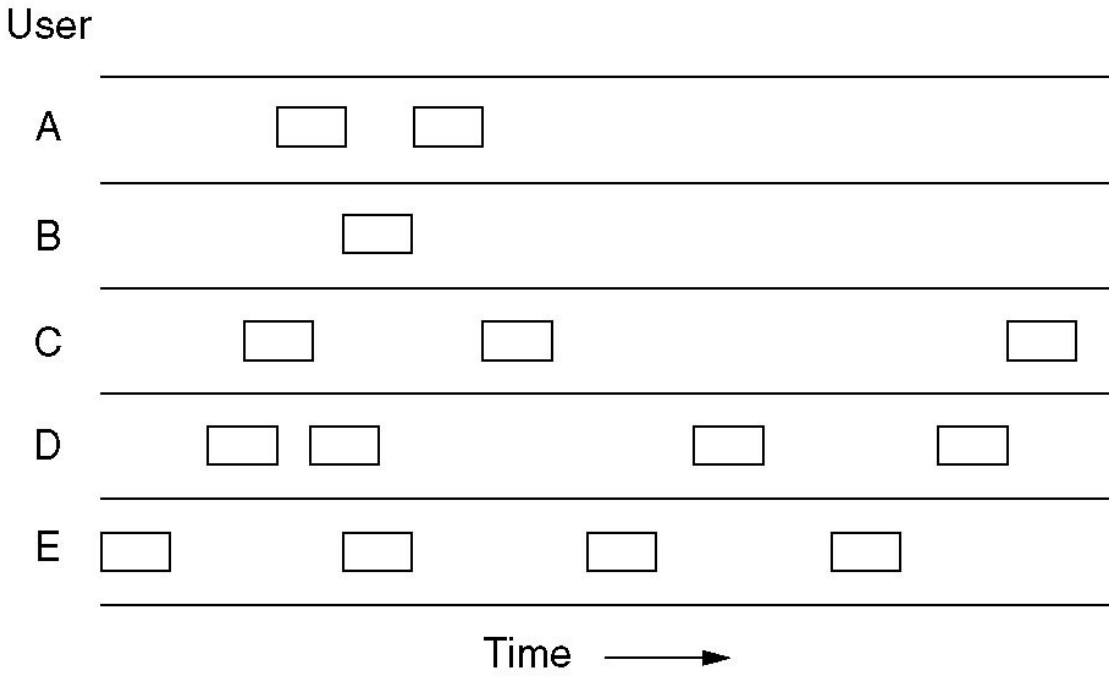
- ALOHA Protocols
  - Pure ALOHA
  - Slotted ALOHA
- CSMA
- CSMA/CD

1) ALOHA Protocols

The core idea is extremely simple:

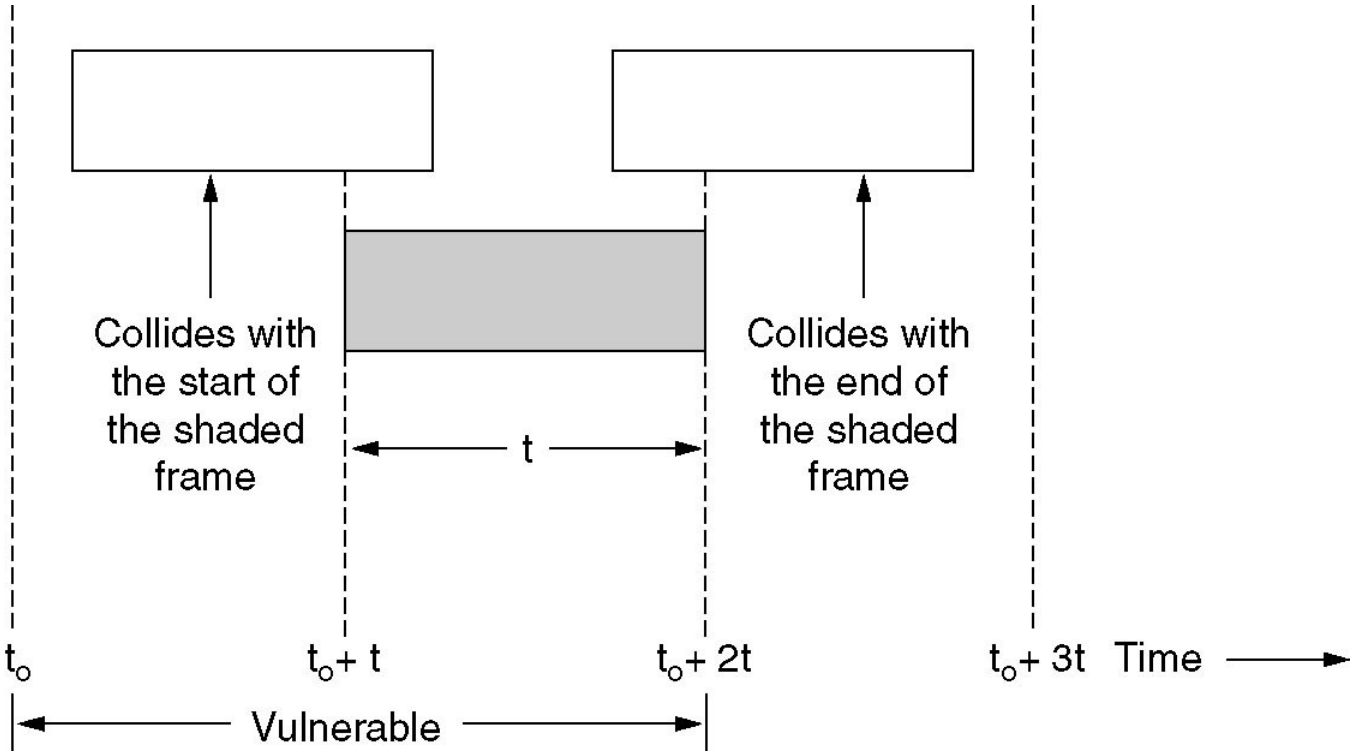
- Anyone may transmit whenever they want. (Continuous time model.)
- Detect if the transmission is successful. (So we need some way for Collision Detection (CD)).
- After a collision, wait a random amount of time and transmit the same frame again. This technique is known as *backoff*.

1.1) Pure ALOHA (1)



- In pure ALOHA, frames are transmitted at completely arbitrary times.
- Vulnerable to collisions → max throughput: **~18.4%**

1.1) Pure ALOHA (2)

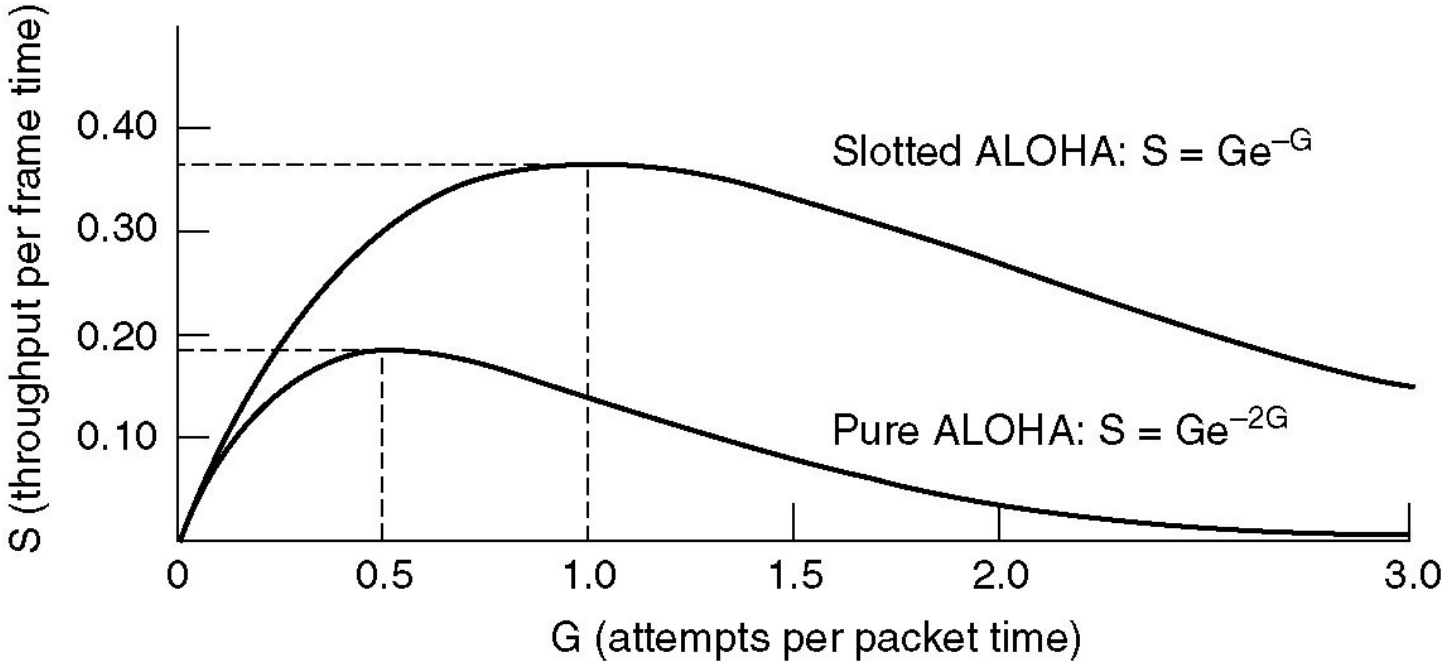


- Vulnerable period for the shaded frame.

1.2) Slotted ALOHA

- Time is divided into slots… can only transmit at the start of slot
- Vulnerable period halved => max. eff is doubled
- Requires sync of clocks
- Still poor at hi-loads

1.2) Slotted ALOHA (2)



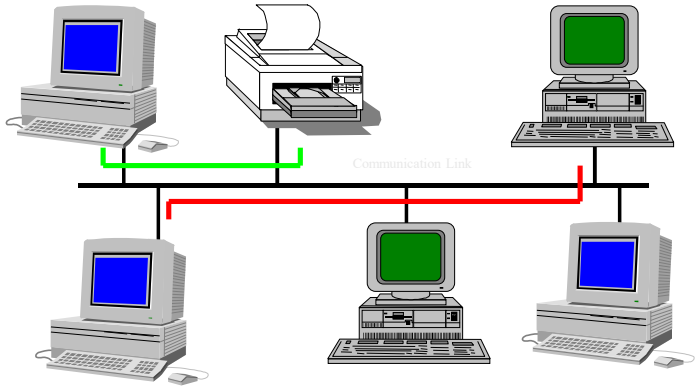
- Throughput versus offered traffic for ALOHA systems.
- Reduces collision window → max throughput: **~36.8%**

Issue:

•No sensing — just transmits and hopes for success.

•Lots of collisions → inefficient for high traffic

2) CSMA (Carrier Sense Multiple Access) (1)



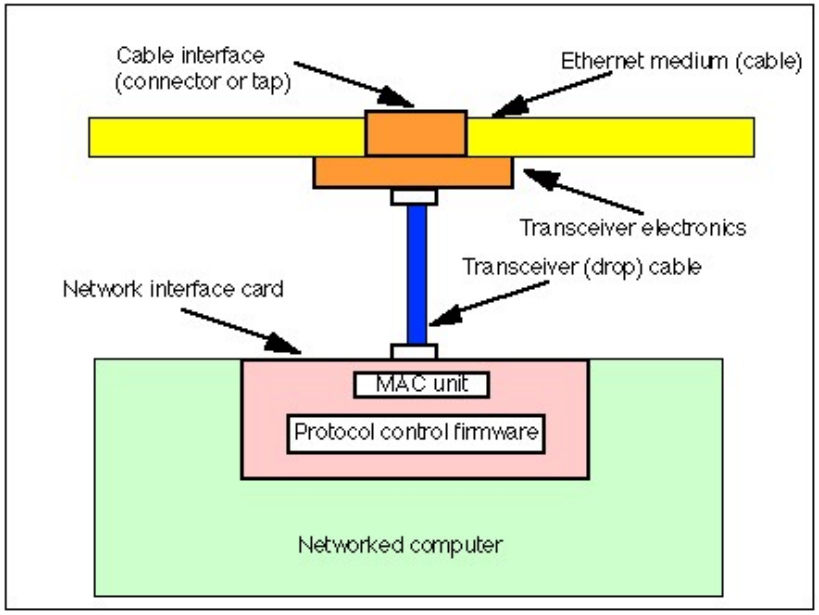
We can improve the performance of our simple network greatly if we introduce **carrier sensing (CS)**. With carrier sensing, each host listens to the data being transmitted over the cable.

- A host will only transmit its own frames when it cannot hear any data being transmitted by other hosts.
- When a frame finishes, an *interframe gap* is allowed to pass before another host starts transmitting its frame.

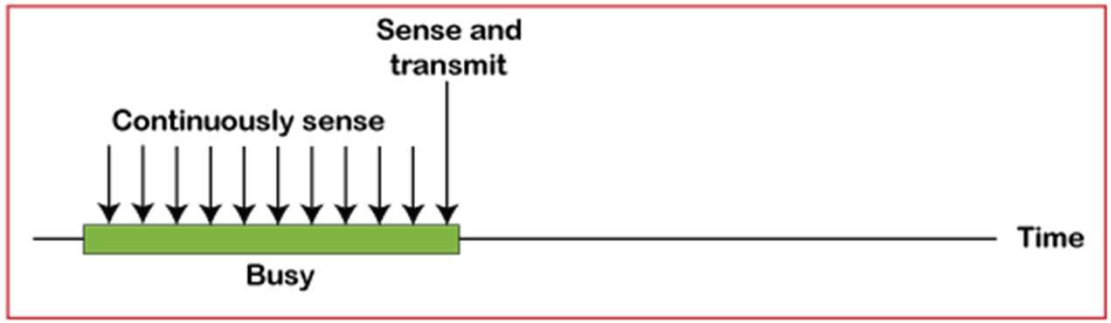
2) CSMA (2)

Improves performance when higher medium utilisation

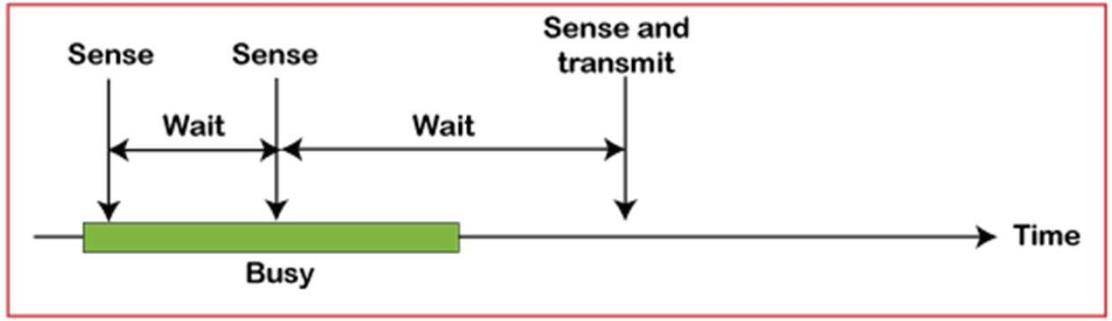
When a node has data to transmit, the node ***first*** listens to the cable (using a transceiver) to see if a carrier (signal) is being transmitted by another node.



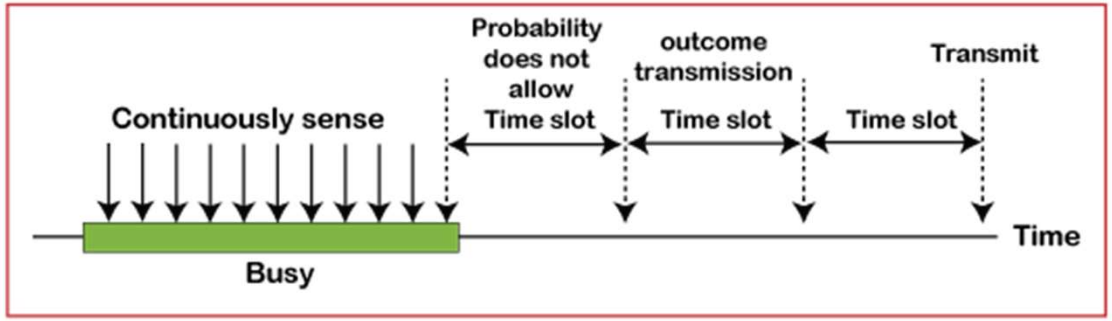
CSMA Types



a. 1-persistent



b. Nonpersistent



c. p-persistent

Carrier Sense Multiple Access (CSMA)

- Carrier Sense Multiple Access (CSMA)
- is a network protocol that manages how network devices share the use of a communication channel.
- There are different types of CSMA protocols, each dealing with collision detection and avoidance differently.
- https://www.youtube.com/watch?v=iKn0GzF5-IU

1-persistent CSMA

- **How it works:** Devices continuously sense the carrier and transmit as soon as the channel becomes idle. If the channel is busy, they wait until it becomes idle. This can lead to high collision rates in networks with high traffic.
- **Strengths:** Fast response once the channel becomes available.
- **Weaknesses:** High probability of collision, especially if multiple devices are waiting.

Non-persistent CSMA

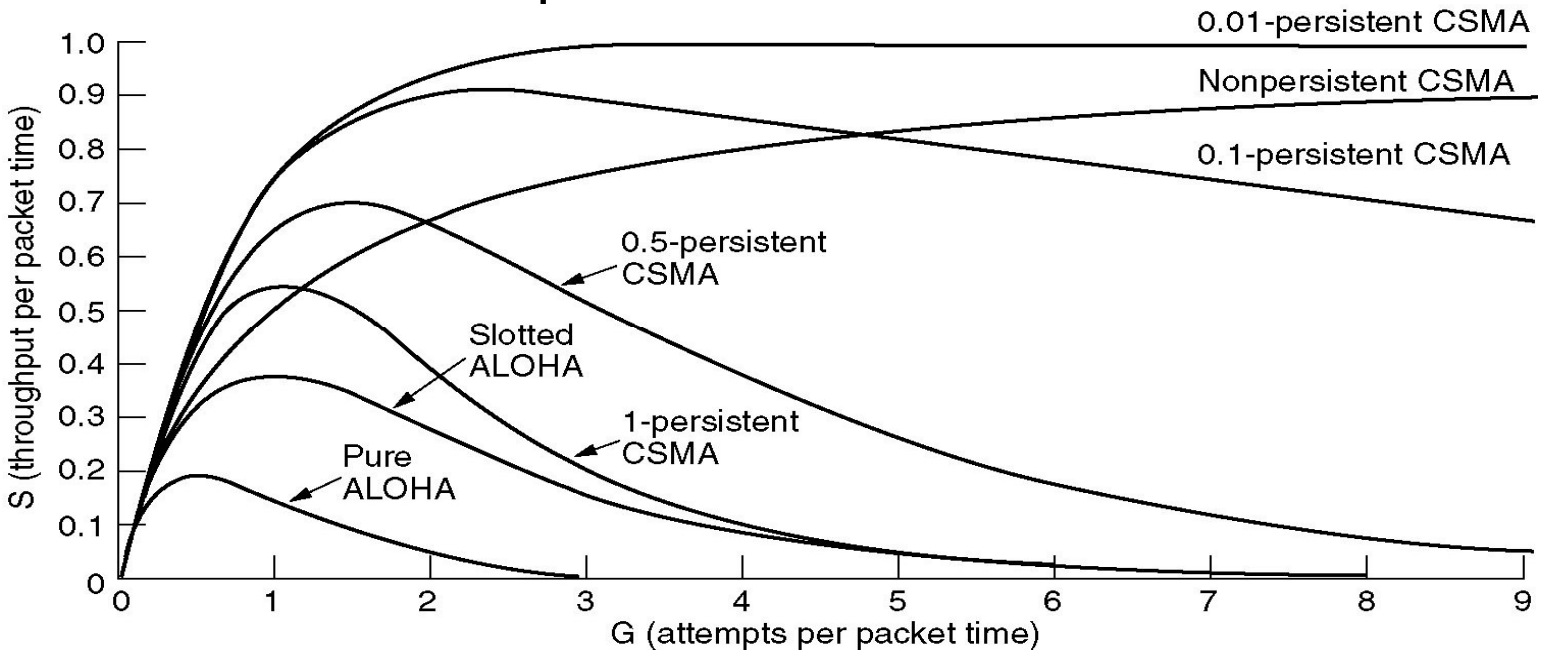
- **How it works:** Devices sense the carrier, and if the channel is busy, they wait for a random period before checking again. This reduces the chance of collisions but adds delay.
- **Strengths:** Reduces the number of collisions by introducing random wait times.
- **Weaknesses:** Can introduce unnecessary delays, especially in low-traffic situations.

p-persistent CSMA

- **How it works:** Devices sense the channel and transmit with a certain probability ppp when the channel is idle. If they don’t transmit, they wait for the next time slot and repeat the process until they either transmit or detect the channel is busy.
- **Strengths:** Balances between collision reduction and transmission delay.
- **Weaknesses:** Not optimal in highly congested networks, but performs well in moderate traffic scenarios.

CSMA (3): Persistent and Nonpersistent CSMA

- Comparison of the channel utilization versus load for various random access protocols.



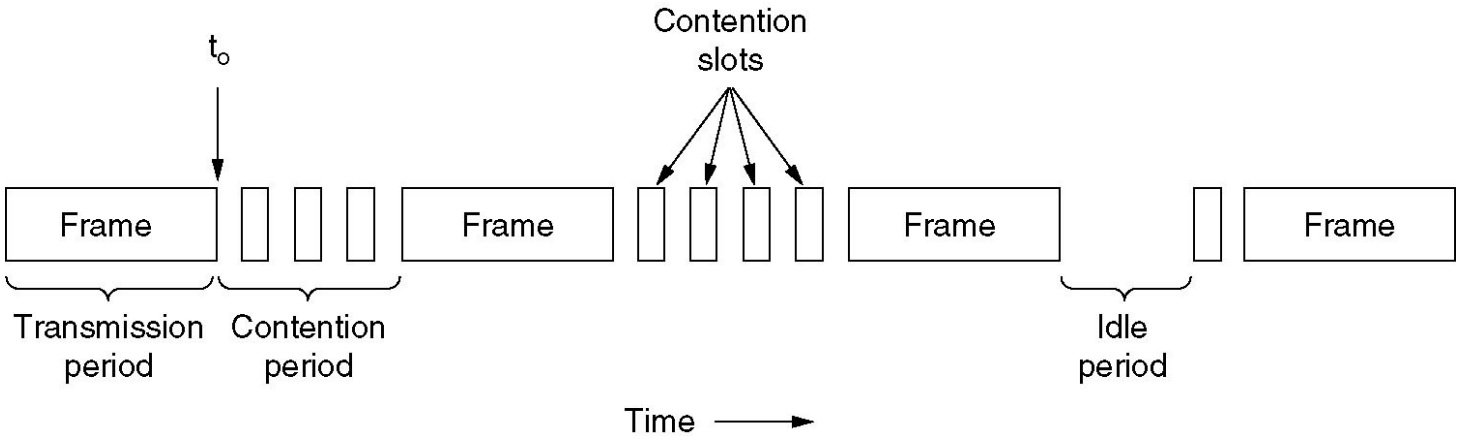
CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)

- **Used in:** Wireless networks (e.g., Wi-Fi)
- **How it works:** Devices first check if the channel is clear. If it is, they signal their intent to transmit data (through a "Request to Send" message), and only transmit if they receive permission (via a "Clear to Send" message). This minimizes the chance of collisions.
- **Strengths:** Suitable for wireless networks where it's harder to detect collisions.
- **Weaknesses:** Overhead increases due to the additional signaling for collision avoidance.

CSMA/CD (Carrier Sense Multiple Access with Collision Detection)

- **Used in:** Wired Ethernet (mostly legacy)
- **How it works:** Devices check if the channel is clear before transmitting data. If a collision occurs (two devices send data simultaneously), they stop transmitting, wait for a random backoff time, and then try again.
- **Strengths:** Suitable for environments where collisions can be detected easily (e.g., wired connections).
- **Weaknesses:** Collisions still happen, and recovery adds to overhead, especially in high-traffic networks.

CSMA with Collision Detection (CSMA/CD) (1)



- CSMA/CD can be in one of three states: contention, transmission, or idle.

CSMA/CD : Ethernet MAC Sublayer Protocol (Minimum Frame Length)

Example #1: Cable = 400m, trans. speed = 10 Mbit/sec, propagation speed = 2*10^8 m/sec

$$t_{prop} = \frac{d}{V} = \frac{400}{2 \times 10^{8}} = 2 \times 10^{-6} = 2\ \mu\ \text{sec}$$

$$2 \times t_{prop} = 4\ \mu\ \text{sec}$$

$$R = 10\ Mbps$$

$$t_b = \frac{1}{R} = \frac{1}{10,000,000} = 0.1\ \mu\ \text{sec}$$

$$n_b = 2 \times \frac{t_p}{t_b} = \frac{4}{0.1} = 40\ bits$$

A margin of error is usually added to this (often to make it a power of 2) so we might use 64 bits (8 bytes).

CSMA/CD : Ethernet MAC Sublayer Protocol (Minimum Frame Length)

Example 2: Speed transmission is 100 Mbits/sec; frame size is 1500 bytes; the propagation speed is 3*10^8 m/sec.

Calculate the distance between the nodes such that the

time to transmit the frame = time to recognize that the collision have occurred.

$$t_{frame} = \frac{L}{R} = \frac{1500 \times 8}{100 \times 10^{6}} = 1.2 \times 10^{-4}$$

$$T_{round_trip} \rightarrow t_{frame} = 2 \times t_{prop}$$

$$t_{prop} = \frac{t_{frame}}{2} = \frac{1.2 \times 10^{-4}}{2} = 6 \times 10^{-5}$$

$$t_{prop} = \frac{d}{V}$$

$$d = t_{prop} \times V = \left(6 \times 10^{-5}\right) \times \left(3 \times 10^{8}\right) = 18 \times 10^{3} = 18 \quad km$$

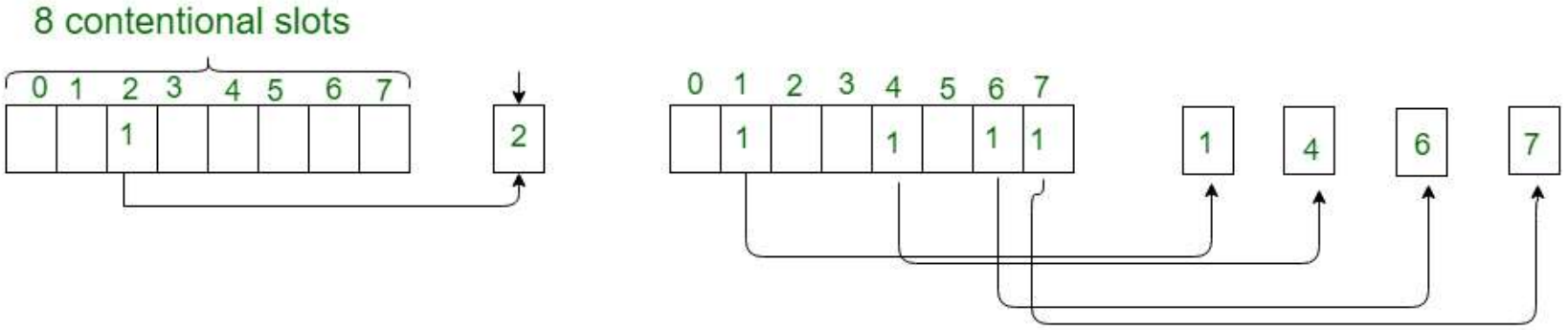
Comparison Summary:

- **CSMA/CD** is suited for wired networks, and CSMA/CA is commonly used in wireless environments to prevent collisions before they occur.
- The persistence methods (1-persistent, non-persistent, p-persistent) modify how devices behave when they sense the channel, influencing the trade-off between collision likelihood and waiting time.
- Each of these CSMA variants is tailored for different types of communication environments and traffic conditions.

Collision Free Protocols

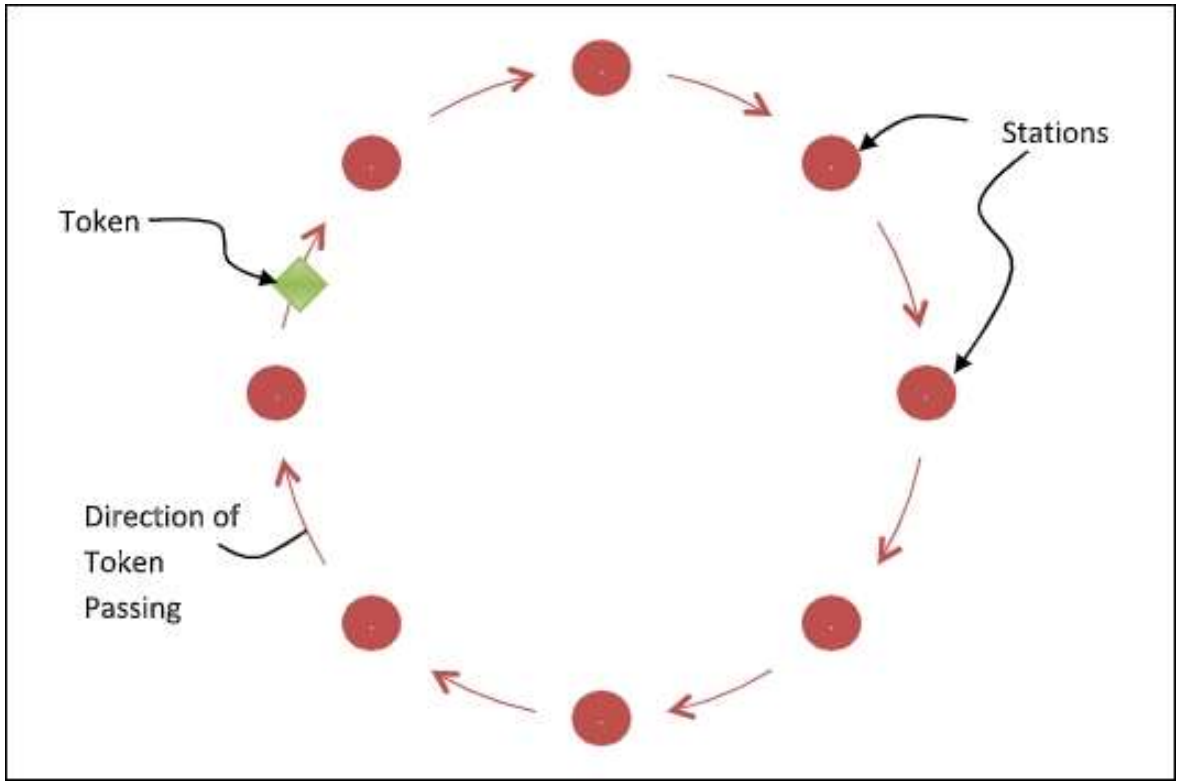
- Bit map Protocol
- Token Passing
- Binary Countdown

Bit Map



- Protocols that broadcast the desire to transmit, before actual transmission are called Reservation Protocols
- Channel ownership is reserved in Advance
- Prevents collision

Token Passing



Binary Countdown

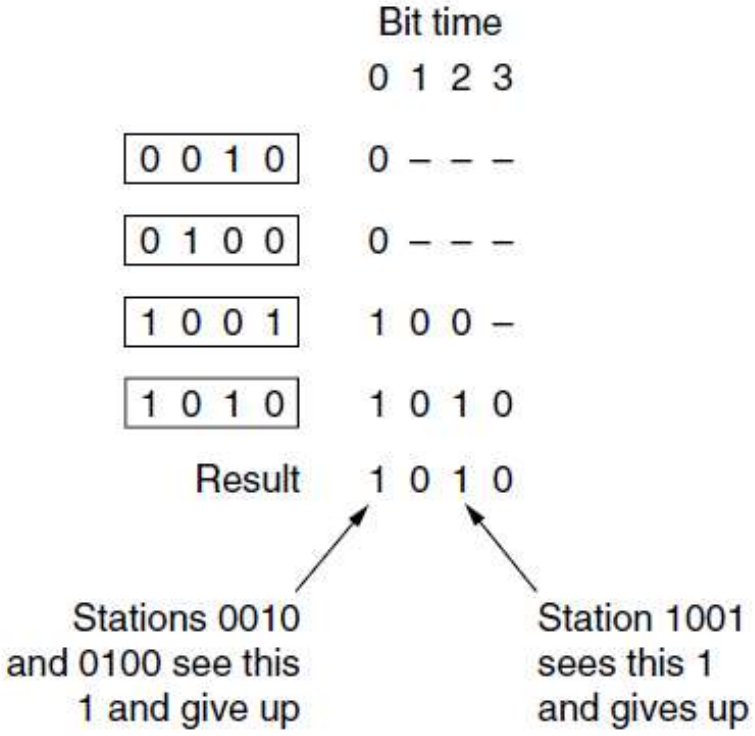


Figure 4-8. The binary countdown protocol. A dash indicates silence.

Wireless LANs

Wireless LAN Protocols

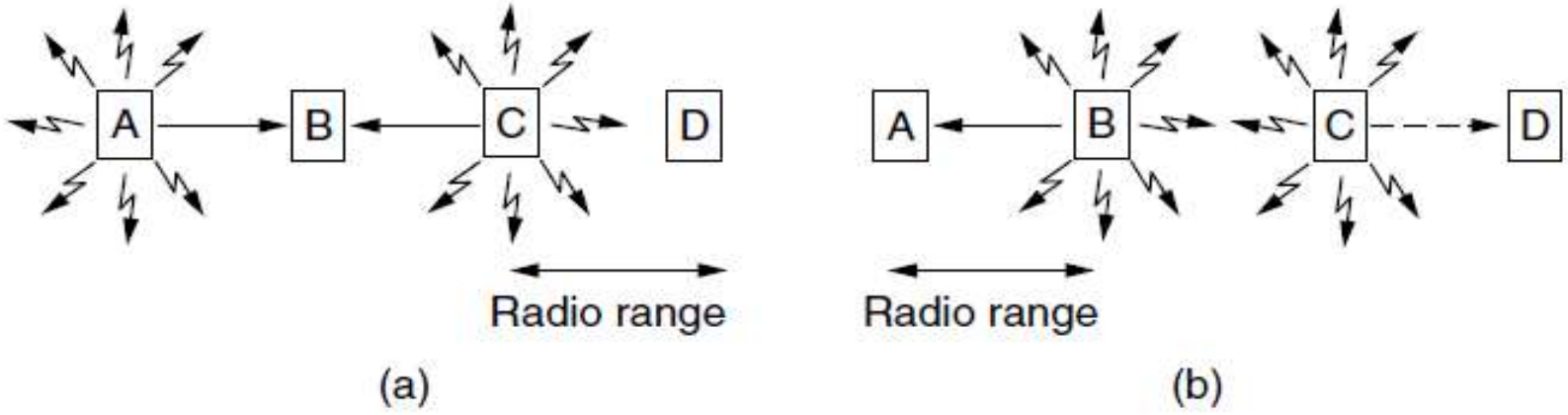


Figure 4-11. A wireless LAN. (a) *A* and *C* are hidden terminals when transmitting to *B*. (b) *B* and *C* are exposed terminals when transmitting to *A* and *D*.

The problem of a station not being able to detect a potential competitor for the medium because the competitor is too far away is called the **HIDDEN TERMINAL PROBLEM.**

EXPOSED TERMINAL PROBLEM

Wireless LAN Protocols

- The difficulty is that, before starting a transmission, a station really wants to know whether there is radio activity around the receiver.
- CSMA merely tells it whether there is activity near the transmitter by sensing the carrier.
- With a wire, all signals propagate to all stations, so this distinction does not exist.

Wireless LANs

MACA-Multiple Access with Collision Avoidance

- sender to stimulate the receiver into outputting a short frame
- so stations nearby can detect this transmission and avoid transmitting for the duration of the upcoming (large) data frame.

  - **Request To Send**
  - **Clear To Send**

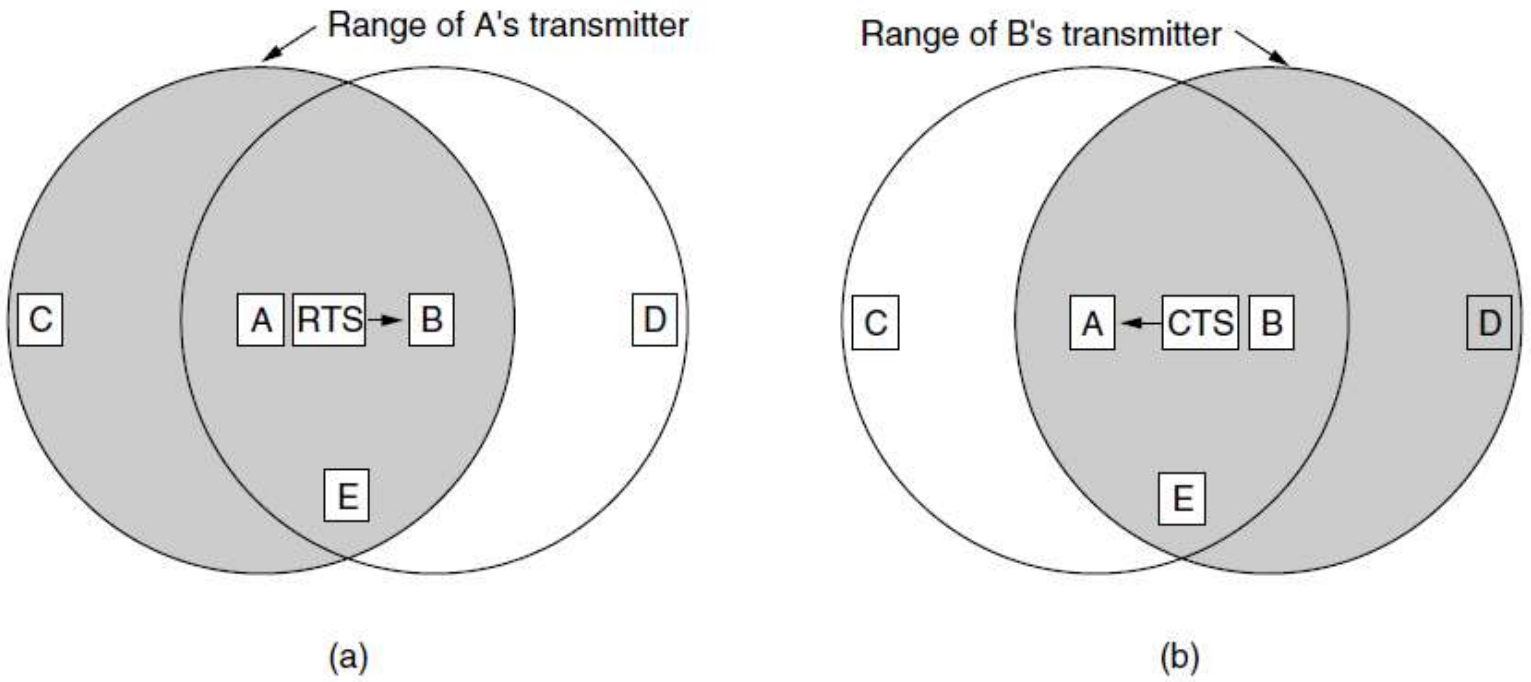


Figure 4-12. The MACA protocol. (a) *A* sending an RTS to *B*. (b) *B* responding with a CTS to *A*.

- RTS-short frame (30 bytes) contains the length of the data frame
- CTS-the data length (copied from the RTS frame).

Data Link Layer Switching

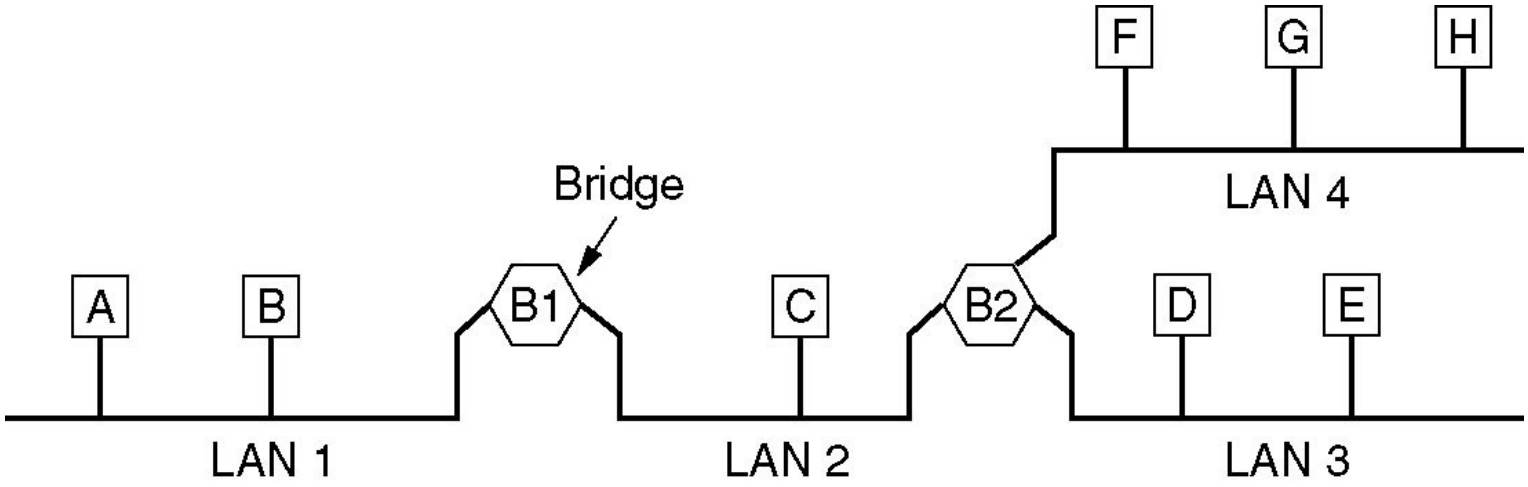
- Uses of Bridges
- Learning Bridges
- Spanning Tree Bridges
- Repeaters, Hubs, Bridges, Switches, Routers, and Gateways
- Virtual LANs

Ethernet Bridge

- Join two LAN segments (A,B), constructing a larger LAN
- Filter traffic passing between the two LANs and may enforce a security policy separating different work groups located on each of the LANs.

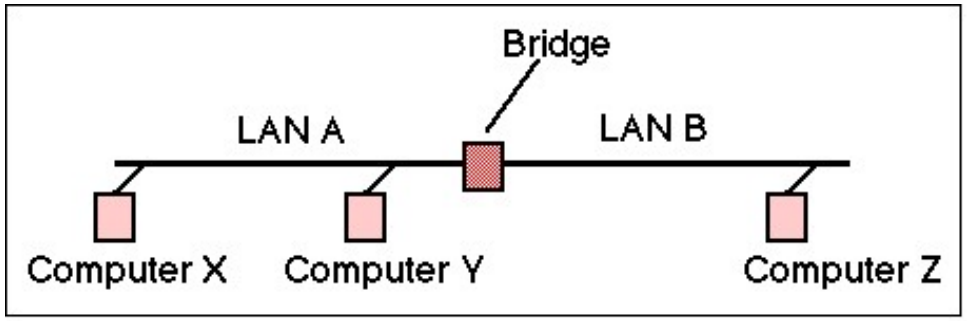
Local Internetworking

- A configuration with four LANs and two bridges.



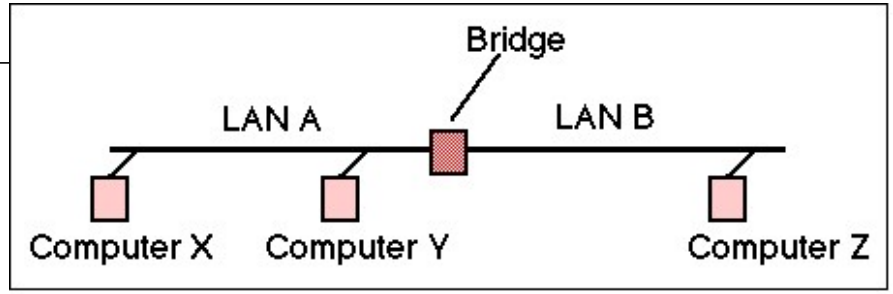
Ethernet Bridges

- Simplest and most frequently used → Transparent Bridge (meaning that the nodes using a bridge are unaware of its presence).

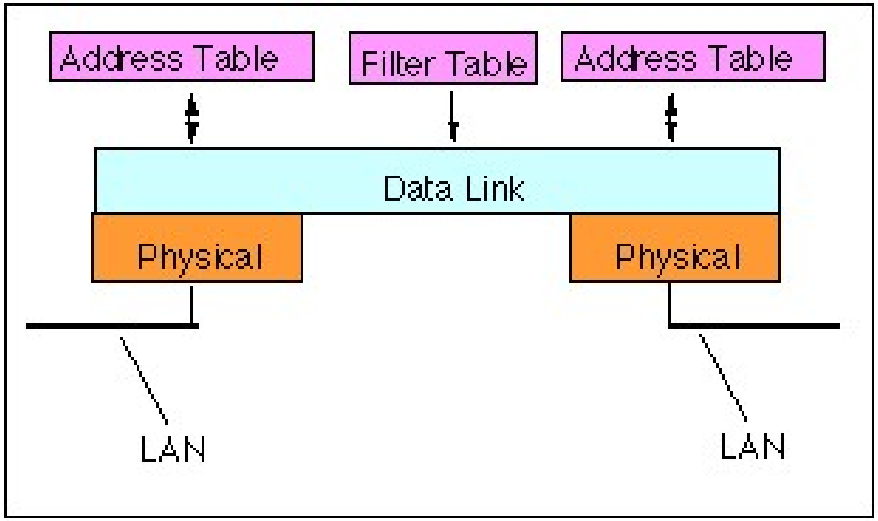


- Bridge could forward all frames, but then it would behave rather like a repeater
- Bridges are smarter than repeaters!

Ethernet Bridges



A bridge stores the hardware addresses observed from frames received by each interface and uses this information to learn which frames need to be forwarded by the bridge.



Ethernet Switch → Modern LANs

- Fundamentally similar to a bridge
- Supports a larger number of connected LAN segments
- Richer management capability.
- Logically partition the traffic to travel only over the network segments on the path between the source and the destination (reduces the wastage of bandwidth)

Ethernet Switch → Benefits

- Improved security
  - users are less able to tap-in into other user's data
- Better management
  - control who receives what information (i.e. Virtual LANs)
  - limit the impact of network problems
- Full duplex
  - rather than half duplex required for shared access

Switched LAN

- **Hub and Switched LAN**
  - hub simulates a single shared medium
  - switch simulates a bridged LAN with one computer per segment

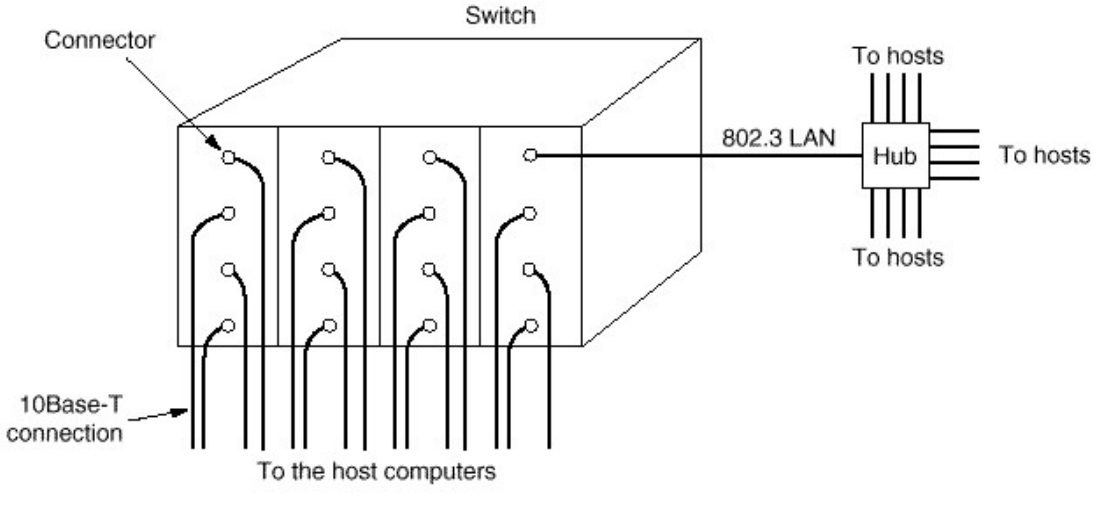


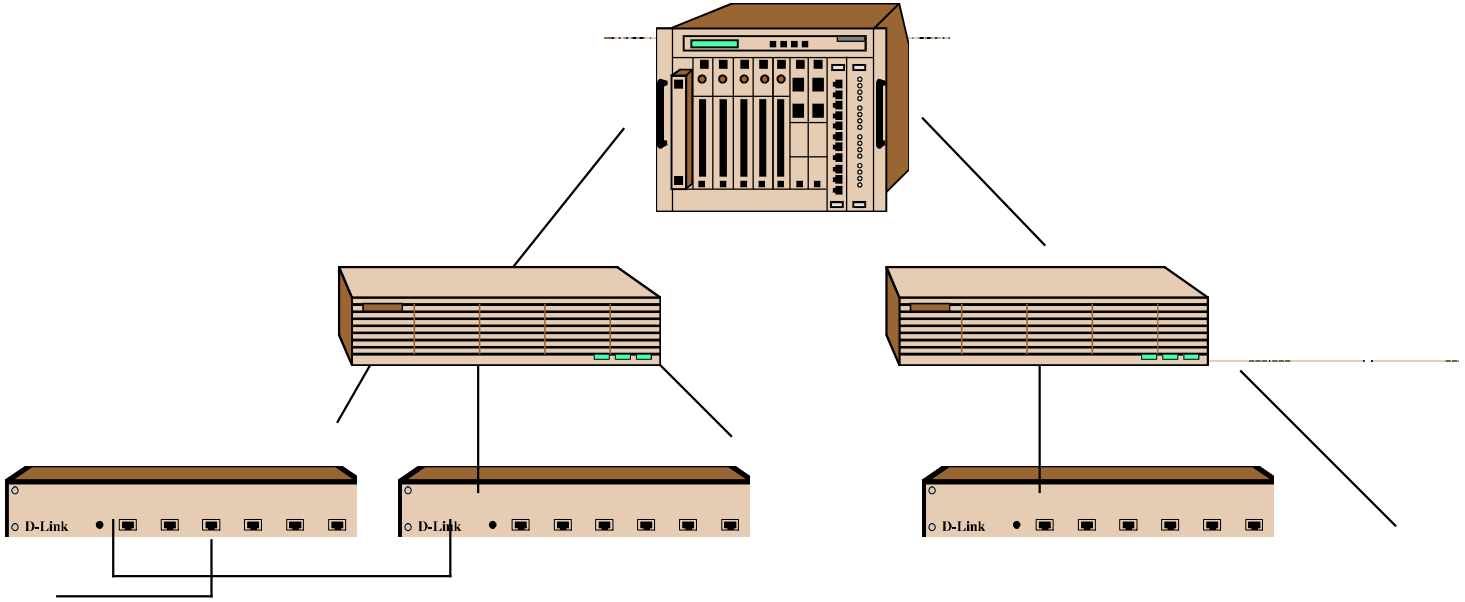
Fig. 4-24. A switched 802.3 LAN.

Ethernet Switches

- Highly Scalable
- 10Base-T switches
  - Competitive with 100Base-TX hubs in both cost and throughput
  - Increasingly used to desktops
- 100Base-TX switches
  - Higher performance (and price)
- Gigabit Ethernet switches
  - Very expensive

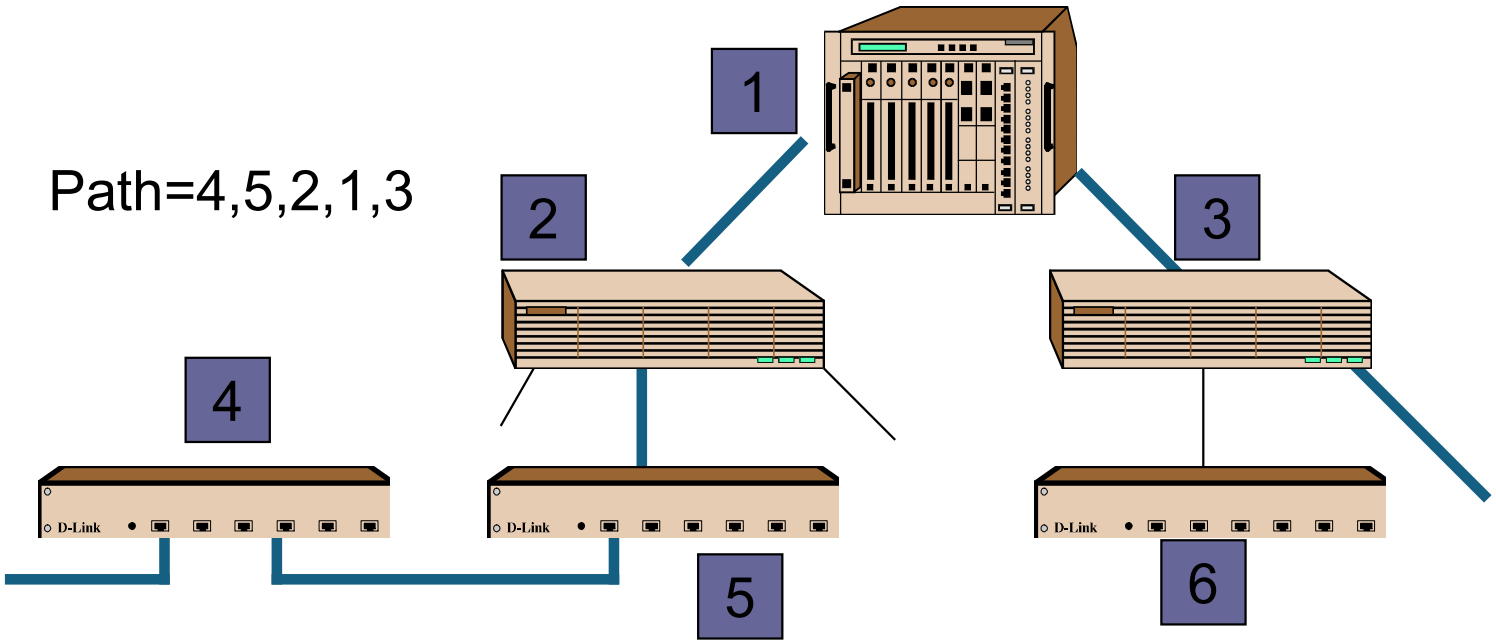
Ethernet Switches

- No limit on number of Ethernet switches between farthest stations
  - So no distance limit on size of switched networks

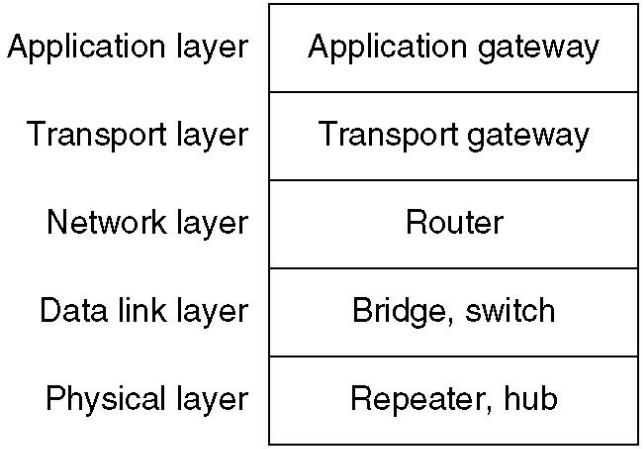


Ethernet Switches

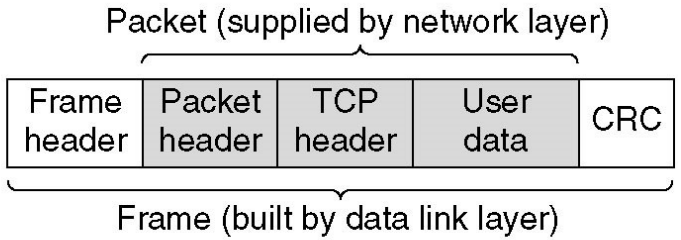
- Ethernet Switches ***must*** be Arranged in a Hierarchy (or daisy chain)
  - Only one possible path between any two stations, switches



Repeaters, Hubs, Bridges, Switches, Routers and Gateways



(a)

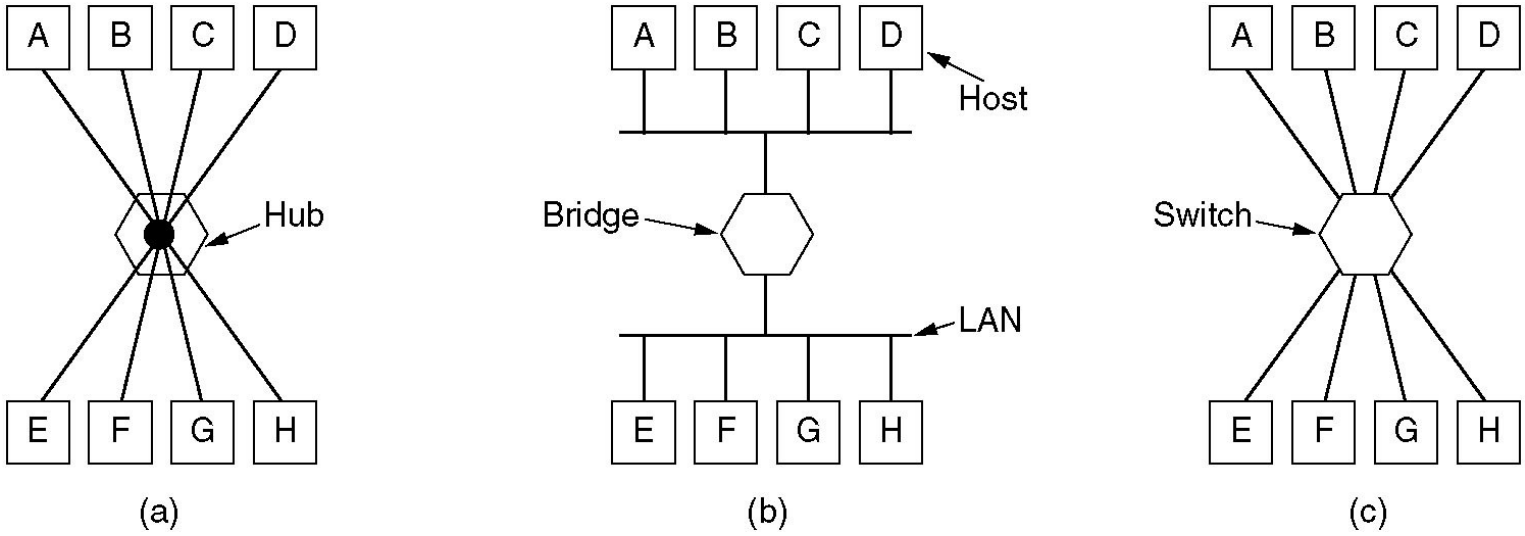


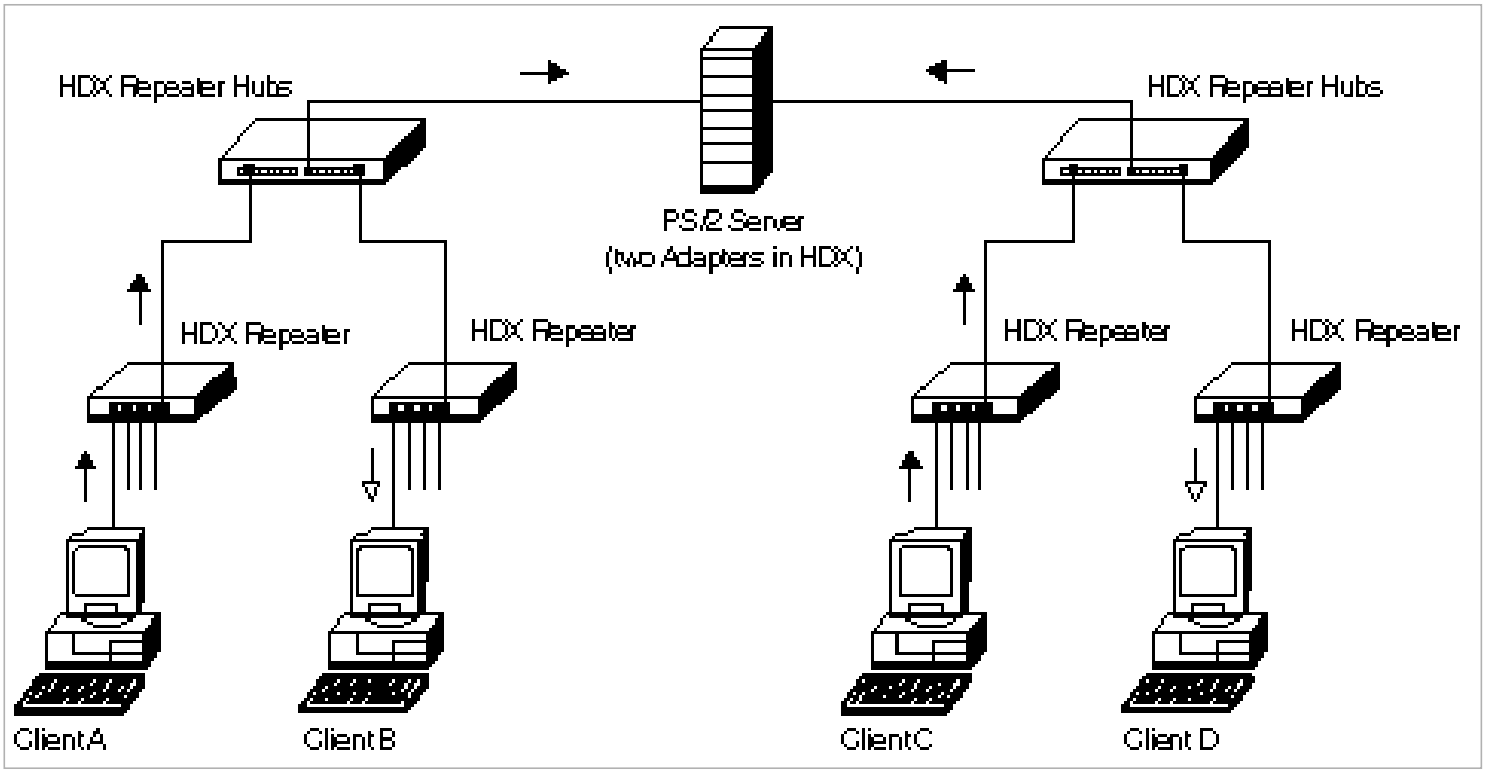
(b)

- (a) Which device is in which layer.
- (b) Frames, packets, and headers.

Repeaters, Hubs, Bridges, Switches, Routers and Gateways

- (a) A hub. (b) A bridge. (c) a switch.



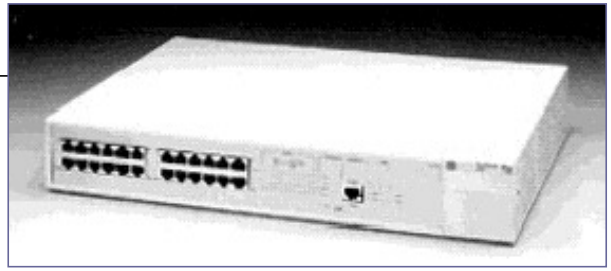


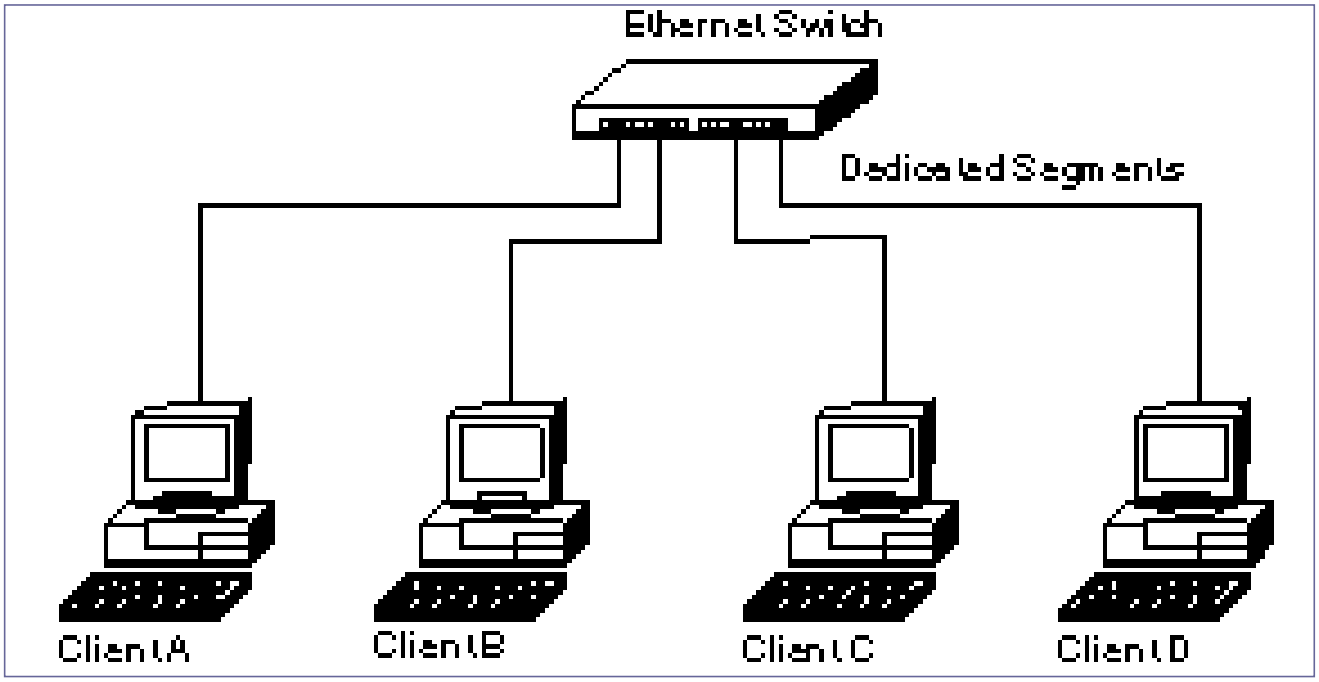


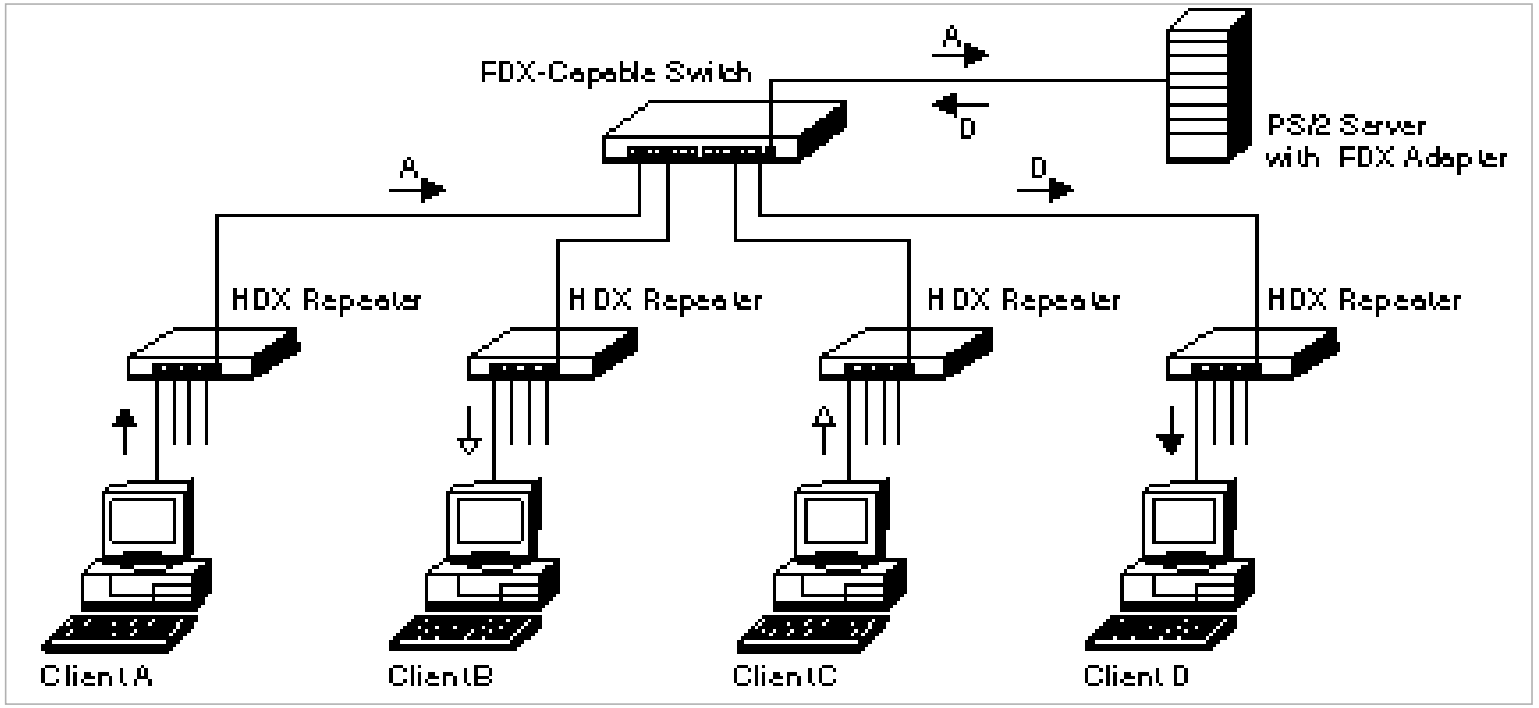
Repeater HUBs

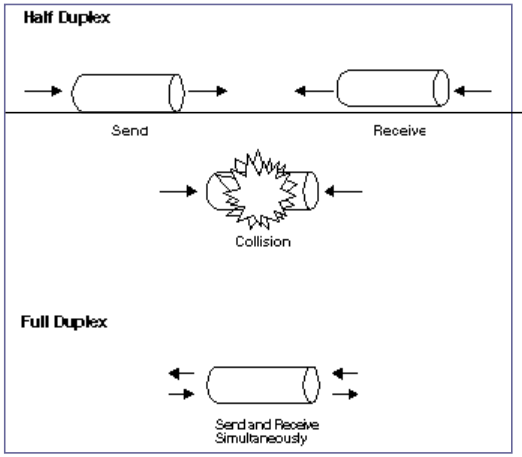


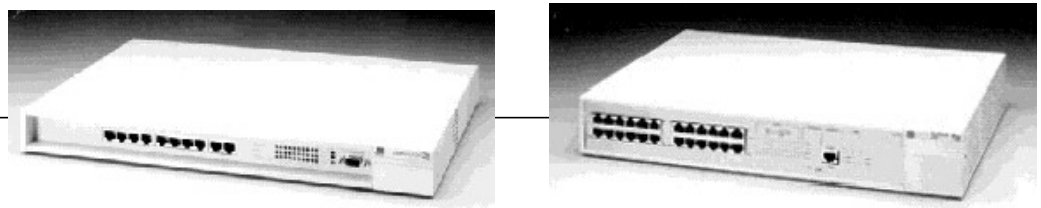
Switches




Ethernet Switch
Dedicated Segments
Client A
Client B
Client C
Client D



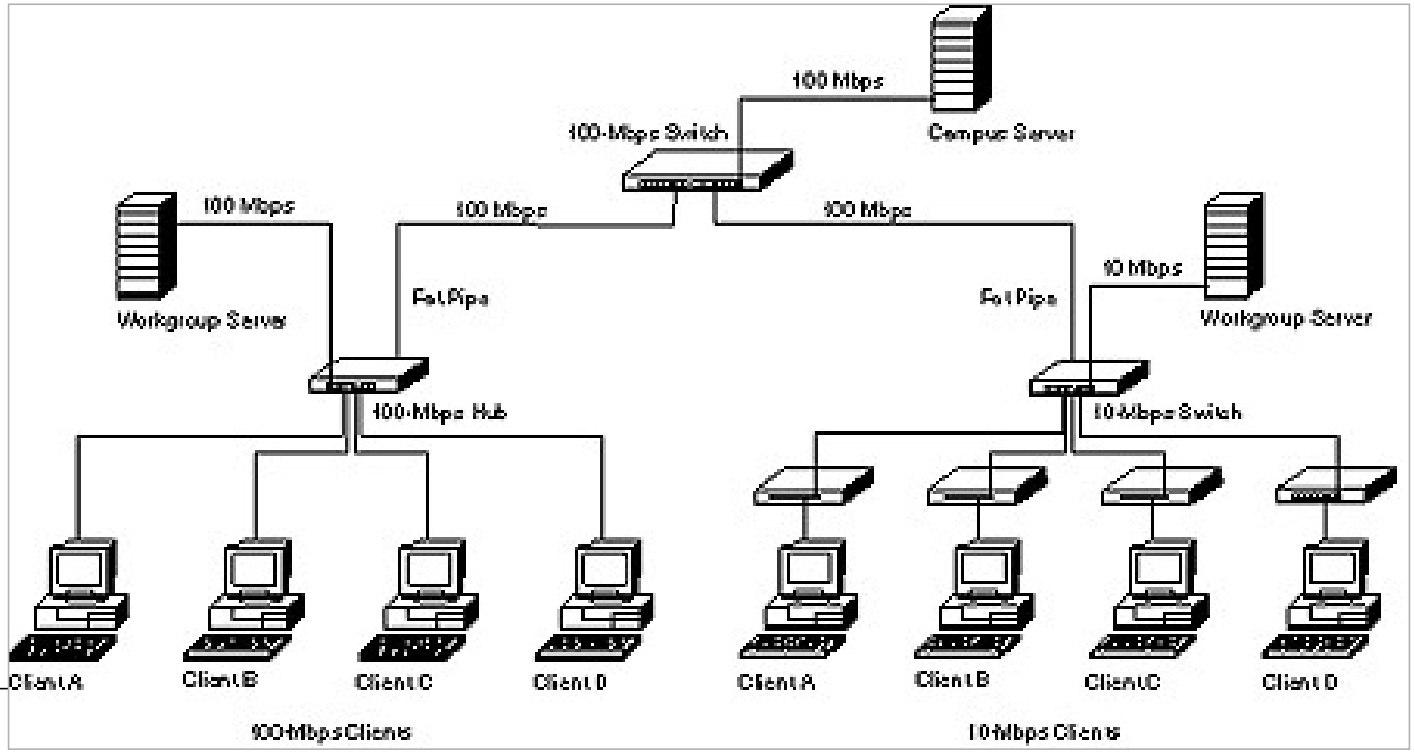


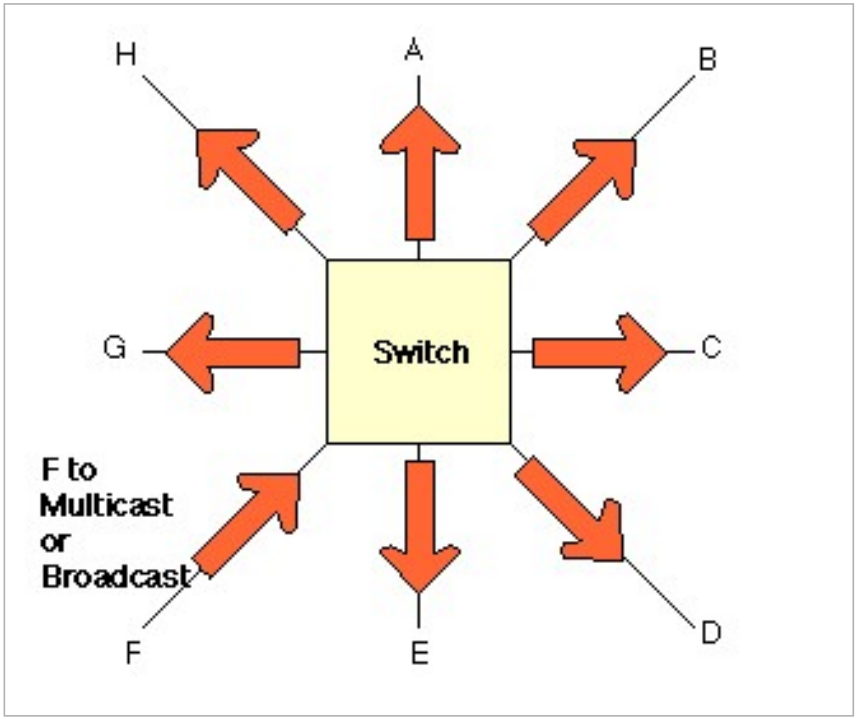


Switches



Repeater HUBs


100 Mbps
Campus Server
100-Mbps Switch
100 Mbps
100 Mbps
100 Mbps
Workgroup Server
Fat Pipe
Fat Pipe
10 Mbps
Workgroup Server
100-Mbps Hub
10-Mbps Switch
Client A
Client B
Client C
Client D
Client A
Client B
Client C
Client D
100-Mbps Clients
10-Mbps Clients



Multicast Traffic from F is delivered to all output interfaces (ports) which asks for it

Switches Versus Routers

- Switches
- Fast
- Inexpensive
- No benefits of alternative routing
- No hierarchical addressing

- Routers
- Slow
- Expensive
- Benefits of alternative routing
- Hierarchical addressing

“Switch where you can; route where you must”